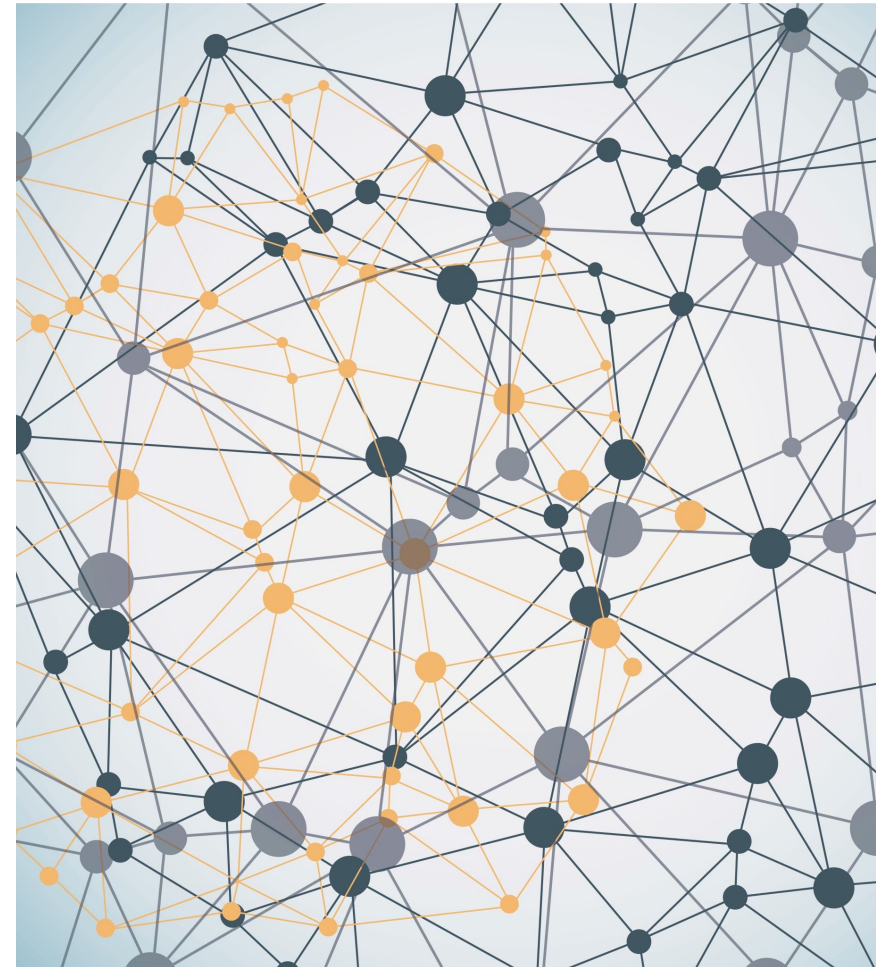


Natural Language Processing

Chapter 1: Introduction

PhD. Nguyen Thi Mai Trang



Content



Introduction



NLP and its applications



NLP pipeline



Ambiguity



Some brief history



Conclusion

What is NLP?

Natural Language Understanding

- Taking some spoken/typed sentence and working out what it means

Natural Language Generation

- Taking some formal representation of what you want to say and working out a way to express it in a natural human language like English

Fundamental goal: deep understand of broad language

- Not just string processing or keyword matching

Target end systems

- speech recognition, machine translation, question answering...
- spelling correction, text categorization...

Languages

- The diversity of human languages
- Unique characteristics of each language
- Challenges for NLP across languages

NLP Applications

- Text Categorization - classify documents Text-to-speech by topics, language, author, spam filtering, sentiment classification (positive, negative)
- Spelling & Grammar Corrections
- Speech Recognition
- Summarization
- Question Answering
- Better search engines
- Machine aided translation
- Information Retrieval
Selecting from a set of documents the ones that are relevant to a query
- Extracting data from text
Converting unstructured text into structure data

Top Natural Language Processing Examples Businesses are Employing



**Smart
Assistants**



**Search
Results**



**Predictive
text**



**Language
Translations**



**Digital
Phone Calls**



**Text
Analytics**



**Virtual
Assistants**



**Detecting
Duplications**



**Social Media
Monitoring**



**Marketing
Strategies**

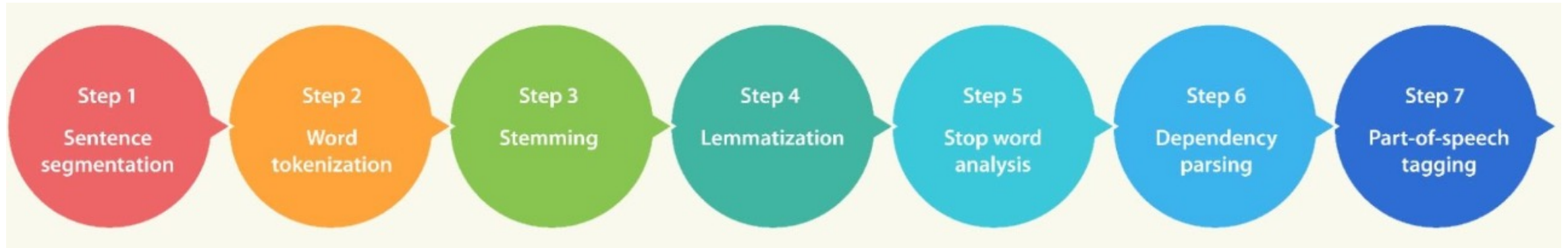


**Descriptive
Analytics**



**Automatic
Insights**

NLP Pipeline



Step 1: Sentence segmentation

Sentence segmentation is the first step in the NLP pipeline. It divides the entire paragraph into different sentences for better understanding.

Hanoi is the capital and second-most populous city of Vietnam. It covers an area of 3,359.82 km², and consists of 12 urban districts, one district-level town and 17 rural districts. It is located within the Red River Delta of Northern Vietnam.

After using sentence segmentation

1. Hanoi is the capital and second-most populous city of Vietnam.
2. It covers an area of 3,359.82 km², and consists of 12 urban districts, one district-level town and 17 rural districts.
3. It is located within the Red River Delta of Northern Vietnam.

Step 2: Word tokenization

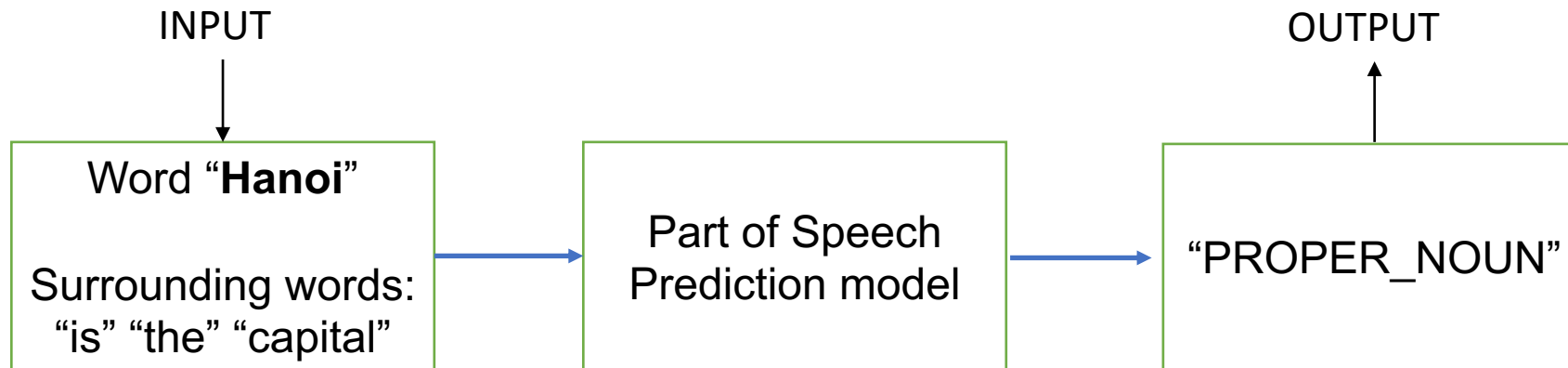
Word tokenization breaks the sentence into separate words or tokens. This helps understand the context of the text

1. Hanoi is the capital and second-most populous city of Vietnam.

"Hanoi" "is" "the" "capital" "and" "second-most" "populous" "city" "of" "Vietnam"

Step 3: Stemming

Stemming helps in preprocessing text. The model analyzes the parts of speech to figure out what exactly the sentence is talking about.



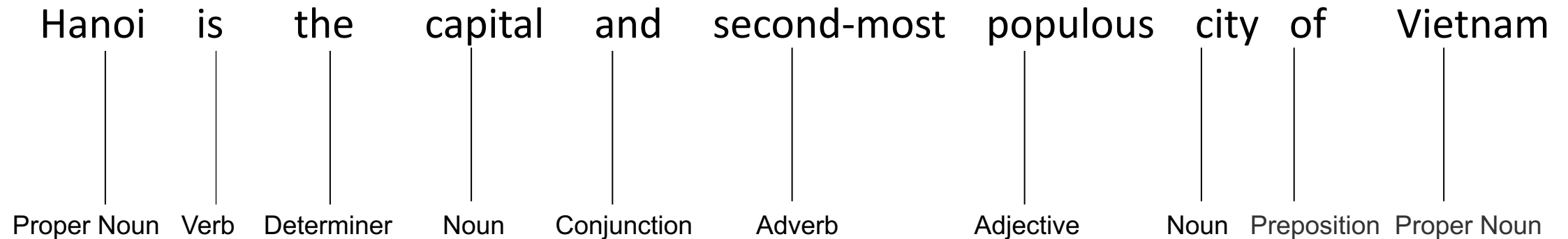
For example:

intelligently *intelligence* *intelligent*

These words originate from a single root word '*intelligen*'

Step 3: Stemming

Stemming normalizes words into their base or root form. In other words, it helps to predict the parts of speech for each token



Step 4: Lemmatization

Lemmatization removes inflectional endings and returns the canonical form of a word or lemma. It is similar to stemming except that the lemma is an actual word.

For example:

‘playing’ and ‘plays’ are forms of the word ‘play’.

Hence, play is the **lemma** of these words.

Unlike a **stem** (recall ‘intelligen’), ‘play’ is a proper word.

Step 5: Stop word analysis

We consider the importance of each and every word in a given sentence.

In English, some words appear more frequently than others such as "is", "a", "the", "and", etc.

As they appear often, the NLP pipeline flags them as stop words. They are **filtered out** so as to **focus on** more important words.

Hanoi **is the** capital **and** second-**most** populous city **of** Vietnam.

Remove stop words

Hanoi capital sencond- populous city Vietnam.

Step 5: Stop word analysis

```
!pip install spacy
!python -m spacy download en_core_web_sm
```

```
import spacy

# Load the spaCy English language model
nlp = spacy.load("en_core_web_sm")

# Original sentence
sentence = "Hanoi is the capital and second-most populous city of Vietnam."

# Process the sentence using spaCy
doc = nlp(sentence)

# Remove stopwords and get non-stopword tokens
filtered_tokens = [token.text for token in doc if not token.is_stop]

# Join the filtered tokens to form the new sentence
filtered_sentence = " ".join(filtered_tokens)

# Print the filtered sentence
print(filtered_sentence)
```



Removing Stopwords
from Sentences with
spaCy in Google Colab

➞ Hanoi capital second - populous city Vietnam .

Step 6: Dependency parsing

Next comes dependency parsing which is mainly used to find out how all the words in a sentence are related to each other. To find the dependency, we can build a tree and assign a single word as a parent word. The main verb in the sentence will act as the root node.

```
import spacy

# Load the spaCy English language model
nlp = spacy.load("en_core_web_sm")

# Original sentence
sentence = "Hanoi is the capital and second-most populous city of Vietnam."

# Process the sentence using spaCy
doc = nlp(sentence)

# Print the dependency tree
for token in doc:
    print(f"{token.text} --{token.dep_}--> {token.head.text}")
```

⇒ Hanoi --nsubj--> is
is --ROOT--> is
the --det--> capital
capital --attr--> is
and --cc--> capital
second --advmod--> populous
- --punct--> most
most --advmod--> populous
populous --amod--> city
city --conj--> capital
of --prep--> city
Vietnam --pobj--> of
. --punct--> is

Step 7: Part-of-speech (POS) tagging

POS tags contain **verbs**, **adverbs**, **nouns**, and **adjectives** that help indicate the **meaning of words** in a grammatically correct way in a sentence.

```
# Original sentence
sentence = "Hanoi is the capital and second-most populous city of Vietnam."

# Process the sentence using spaCy
doc = nlp(sentence)

# Print the POS tags for each word
for token in doc:
    print(f"{token.text}: {token.pos_}")
```

→ Hanoi: PROPN
is: AUX
the: DET
capital: NOUN
and: CCONJ
second: ADV
-: PUNCT
most: ADV
populous: ADJ
city: NOUN
of: ADP
Vietnam: PROPN
.: PUNCT

Ambiguity in NLP

- Natural languages **are ambiguous**, so computers are not able to understand language the way people do.
 - Ambiguity can be referred as the ability of having more than one meaning or being understood in more than one way. Ambiguity is one of those areas of cognitive sciences that **doesn't have a well-defined solution**.
 - The Text based NLP has been regarded as consisting of various levels. They are:
 - Lexical Analysis:- Analysis of word forms
 - Syntactic Analysis:-Structure processing
 - Semantic Analysis:- Meaning representation
 - Discourse Analysis:- Processing of interrelated sentences
 - Pragmatic Analysis:-The purposeful use of sentences in situations.
- Ambiguity can occur at all these levels. It is a property of linguistic expressions.

Ambiguity in NLP

- If an expression (word/phrase/sentence) has more than one interpretation we can refer it as ambiguous.
- For eg:

The chicken is ready to eat

Interpretations: The chicken is ready to be fed or

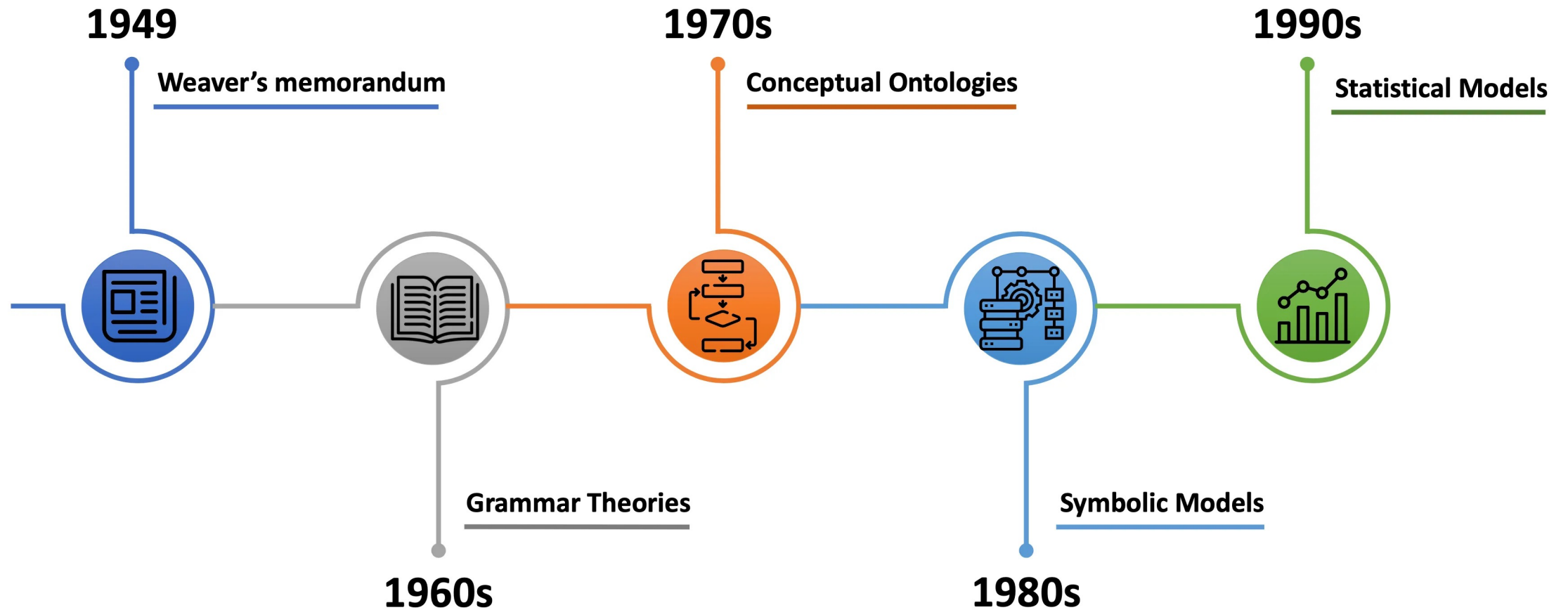
The chicken (food) is ready to be eaten

There was not a single man at the party

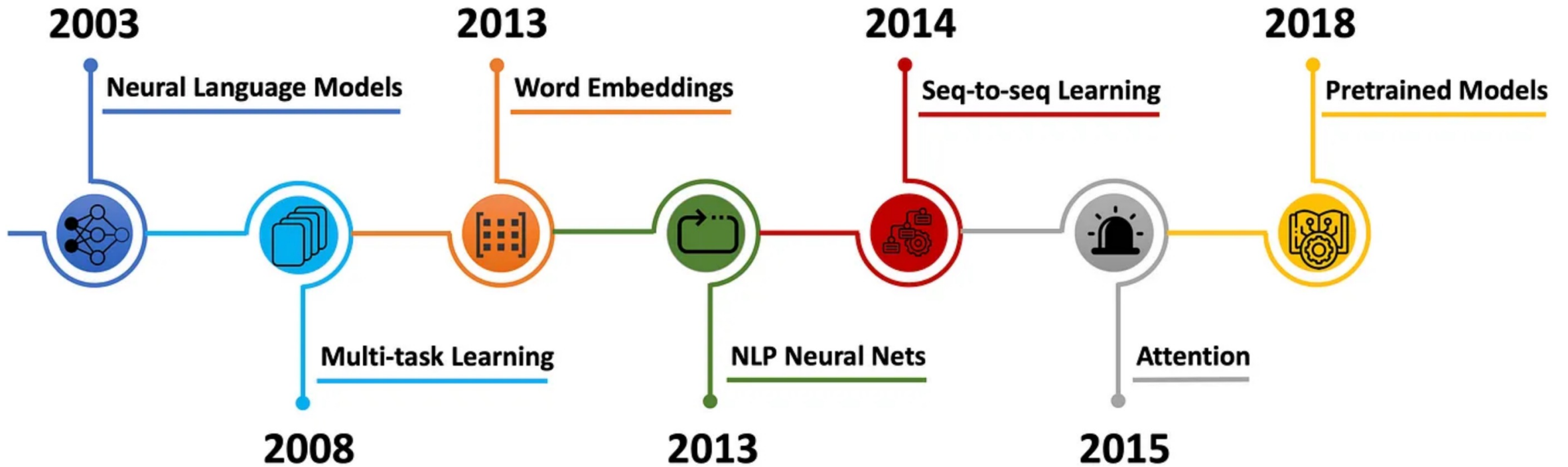
Interpretations: Lack of bachelors (unmarried men) at the party

Lack of men altogether

Brief History of NLP



Brief History of NLP



NLP Challenges

❑ NLP systems needs to answer the question "who did what to whom"

- MANY hidden variables
- Knowledge about the world
- Knowledge about the context
- Knowledge about human communication techniques
 - Can you tell me the time?

❑ Problem of scale

- Many (infinite?) possible words, meanings, context

NLP Challenges

❑ Problem of sparsity

- Very difficult to do statistical analysis, most things (words, concepts) are never seen before

❑ Long range correlations

❑ Key problems

- Representation of meaning
- Language presupposes knowledge about the world.
- Language only reflects the surface of meaning
- Language presupposes communication between people

NLP Challenges

- ❑ Different ways of Parsing a sentence
- ❑ Word category ambiguity
- ❑ Word sense ambiguity
- ❑ Words can mean more than their sum of parts - The Times of India
- ❑ Imparting world knowledge is difficult - the blue pen ate the ice-cream

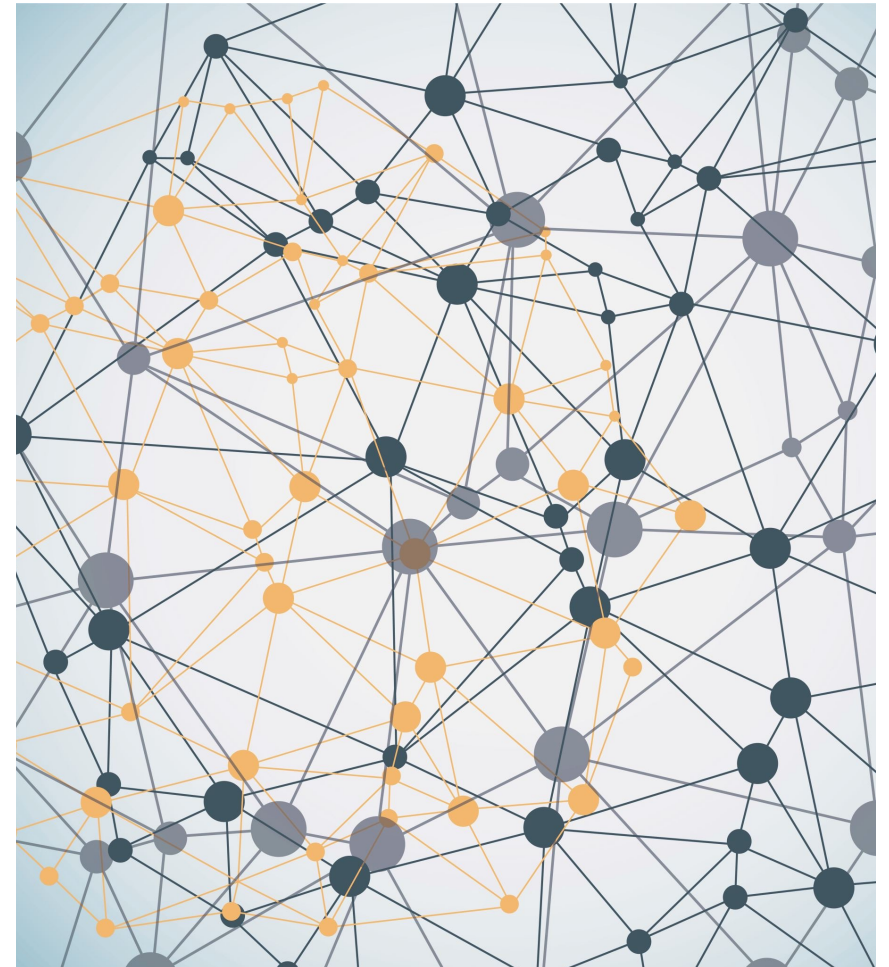
NLP Challenges

- ❑ Fictitious worlds - people on mars can fly
- ❑ Defining scope
 - *people like ice-cream - does this mean all people like ice cream?*
- ❑ Language is changing and evolving
- ❑ Complex ways of interaction between the kinds of knowledge
- ❑ Exponential complexity at each point in using the knowledge

Natural Language Processing

Chapter 2: Part-of-Speech Tagging

PhD. Nguyen Thi Mai Trang



Content

Words and word classes

- Words
- Word classes

The Penn Treebank part-of-speech tagset

Part-of-speech tagging

HMM part-of-speech tagging

- Markov chains
- The Hidden Markov Model (HMM)
- The components of an HMM tagger
- HMM tagging as decoding
- The Viterbi algorithm
- Beam search

WORDS

- What counts as a word?
- **Corpus** (plural **corpora**), a computer-readable collection of text or speech
- Ex: the Brown corpus is a million-word collection of samples from 500 written English texts from different genres (newspaper, fiction, non-fiction, academic, etc.)

How many words are in the following Brown sentence?

He stepped out into the hall, was delighted to encounter a water brother.

=> 13 words if we don't count punctuation marks as words, 15 if we count punctuation

WORDS

- In NLP, we differentiate between types and tokens.
 - Types are distinct words, while tokens are the total number of words, including repetitions.
 - When counting English words, we generally refer to word types (distinct words) rather than tokens (total occurrences).
- Punctuation plays a crucial role in NLP, marking boundaries and conveying meaning.
- Sometimes, in tasks like part-of-speech tagging or parsing, punctuation marks are treated as separate words.
- A lemma is a set of words with the same stem, part-of-speech, and word sense (e.g., "cats" and "cat" share the lemma "cat").
- Wordforms are the full inflected or derived forms of words.

English word classes

Parts of speech fall into two broad categories: closed class and open class

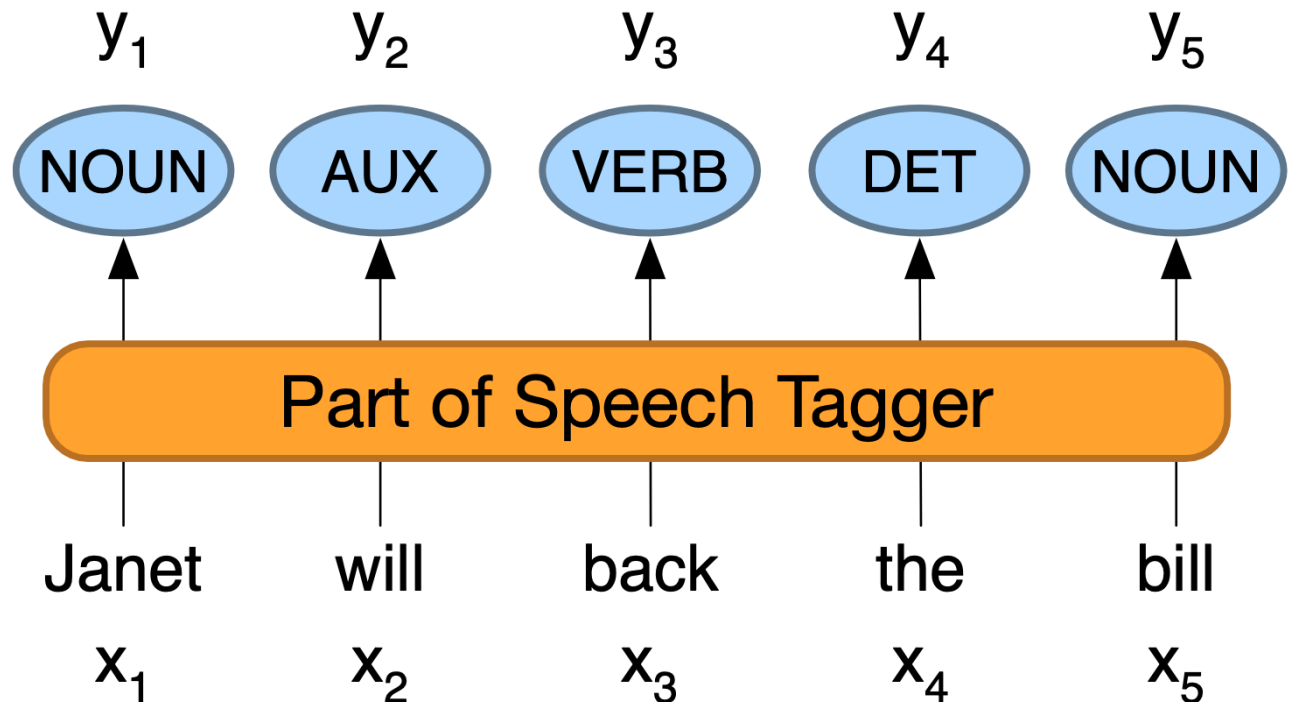
- Closed classes are those with relatively fixed membership, such as prepositions.
- By contrast, nouns and verbs are open classes.

	Tag	Description	Example
Open Class	ADJ	Adjective: noun modifiers describing properties	<i>red, young, awesome</i>
	ADV	Adverb: verb modifiers of time, place, manner	<i>very, slowly, home, yesterday</i>
	NOUN	words for persons, places, things, etc.	<i>algorithm, cat, mango, beauty</i>
	VERB	words for actions and processes	<i>draw, provide, go</i>
	PROPN	Proper noun: name of a person, organization, place, etc..	<i>Regina, IBM, Colorado</i>
	INTJ	Interjection: exclamation, greeting, yes/no response, etc.	<i>oh, um, yes, hello</i>
Closed Class Words	ADP	Adposition (Preposition/Postposition): marks a noun's spacial, temporal, or other relation	<i>in, on, by, under</i>
	AUX	Auxiliary: helping verb marking tense, aspect, mood, etc.,	<i>can, may, should, are</i>
	CCONJ	Coordinating Conjunction: joins two phrases/clauses	<i>and, or, but</i>
	DET	Determiner: marks noun phrase properties	<i>a, an, the, this</i>
	NUM	Numeral	<i>one, two, first, second</i>
	PART	Particle: a function word that must be associated with another word	<i>'s, not, (infinitive) to</i>
	PRON	Pronoun: a shorthand for referring to an entity or event	<i>she, who, I, others</i>
Other	SCONJ	Subordinating Conjunction: joins a main clause with a subordinate clause such as a sentential complement	<i>that, which</i>
	PUNCT	Punctuation	<i>; , ()</i>
	SYM	Symbols like \$ or emoji	<i>\$, %</i>
	X	Other	<i>asdf, qwfg</i>

The 17 parts of speech in the Universal Dependencies tagset (de Marneffe et al., 2021)

Part-of-speech tagging

- Part-of-speech tagging is the process of assigning a part-of-speech to each word in a text.
- The input is a sequence $x_1, x_2, \dots, x_n$ of (tokenized) words and a tagset, and the output is a sequence $y_1, y_2, \dots, y_n$ of tags, each output y_i corresponding exactly to one input x_i ,



Ambiguous

- Tagging is a disambiguation task; words are ambiguous —have more than one possible part-of-speech—and the goal is to find the correct tag for the situation.
- Ex:

Book that fight

Verb

Hand me that book

Noun

Does that fight serve dinner

Determiner

I thought that your fight was earlier

complementizer

- The goal of POS-tagging is to resolve these ambiguities, choosing the proper tag for the context.

The Penn Treebank Part-of-speech Tagset

- The Penn Treebank part-of-speech (POS) tagset is a widely used system for tagging words in a natural language text with their grammatical categories or parts of speech.
- It aimed to annotate a large corpus of English text with syntactic and structural information
- The Penn Treebank POS tagset includes a set of tags, each representing a different part of speech or grammatical category
- Alphabetical list of part-of-speech tags used in the Penn Treebank Project: https://www.ling.upenn.edu/courses/Fall_2003/ling001/penn_treebank_pos.html

Penn Treebank part-of-speech tags

Tag	Description	Example	Tag	Description	Example	Tag	Description	Example
CC	coord. conj.	<i>and, but, or</i>	NNP	proper noun, sing.	<i>IBM</i>	TO	“to”	<i>to</i>
CD	cardinal number	<i>one, two</i>	NNPS	proper noun, plu.	<i>Carolinas</i>	UH	interjection	<i>ah, oops</i>
DT	determiner	<i>a, the</i>	NNS	noun, plural	<i>llamas</i>	VB	verb base	<i>eat</i>
EX	existential ‘there’	<i>there</i>	PDT	predeterminer	<i>all, both</i>	VBD	verb past tense	<i>ate</i>
FW	foreign word	<i>mea culpa</i>	POS	possessive ending	<i>’s</i>	VBG	verb gerund	<i>eating</i>
IN	preposition/ subordin-conj	<i>of, in, by</i>	PRP	personal pronoun	<i>I, you, he</i>	VBN	verb past partici- ple	<i>eaten</i>
JJ	adjective	<i>yellow</i>	PRP\$	possess. pronoun	<i>your, one’s</i>	VBP	verb non-3sg-pr	<i>eat</i>
JJR	comparative adj	<i>bigger</i>	RB	adverb	<i>quickly</i>	VBZ	verb 3sg pres	<i>eats</i>
JJS	superlative adj	<i>wildest</i>	RBR	comparative adv	<i>faster</i>	WDT	wh-determ.	<i>which, that</i>
LS	list item marker	<i>1, 2, One</i>	RBS	superlatv. adv	<i>fastest</i>	WP	wh-pronoun	<i>what, who</i>
MD	modal	<i>can, should</i>	RP	particle	<i>up, off</i>	WP\$	wh-possess.	<i>whose</i>
NN	sing or mass noun	<i>llama</i>	SYM	symbol	<i>+, %, &</i>	WRB	wh-adverb	<i>how, where</i>

Part-of-speech tagging

- Part-of-speech tagging (POS tagging) is the task of tagging a word in a text with its part of speech.

“Hanoi is the capital of Vietnam.”

Now, let's provide the POS tagging for this sentence:

"Hanoi" - Noun (NN)

"is" - Verb (VBZ)

"the" - Article (DT)

"capital" - Noun (NN)

"of" - Preposition (IN)

"Vietnam" - Noun (NN)

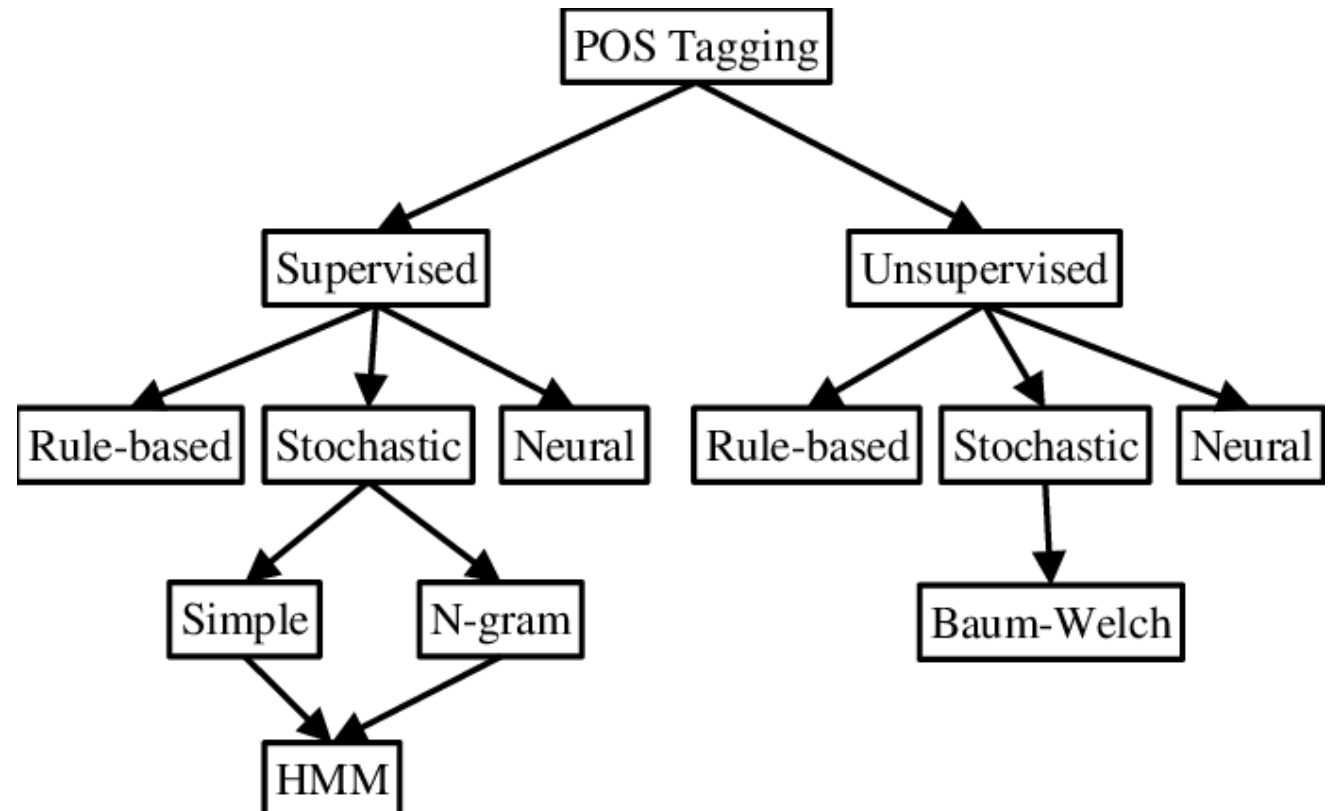
Importance of POS Tagging

- Improves Language Understanding
- Facilitates Syntax Analysis
- Enhances Search and Information Retrieval
- Enables Machine Translation
- Assists Sentiment Analysis

How POS Tagging Works

- There are two main types of POS tagging: **supervised** and **unsupervised**.
- **Supervised POS tagging** methods use a training dataset of labeled text to learn the relationships between words and their POS tags. The training dataset is then used to train a model that can predict the POS tag of a word given its context.
- **Unsupervised POS tagging** methods do not use a labeled training dataset. Instead, they rely on statistical methods to learn the relationships between words and their POS tags. Unsupervised POS tagging methods are often less accurate than supervised methods, but they can be used to tag text in languages where there is no labeled training dataset available.

Methods for implementing POS tagger



HMM part-of-speech tagging

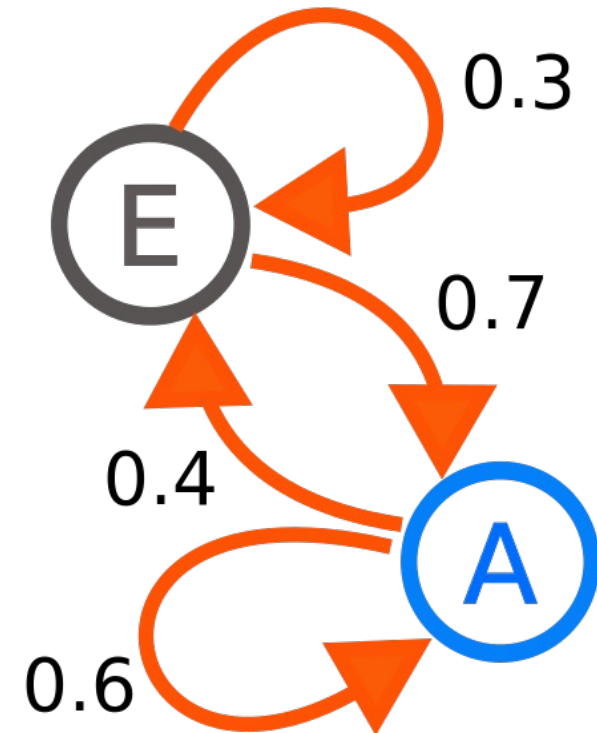
- **Hidden Markov Model** (HMM) part-of-speech (POS) tagging is a statistical method used in natural language processing (NLP) to determine the POS tags for words in a sentence
- HMM POS tagging involves the following key components:
 - Hidden Markov Model
 - Training
 - Tagging Process
 - Decoding

Markov chains

- In 1906, Andrey Markov introduced the Markov chains
- Markov chain is a simple concept which can explain most complicated real time Many of the Artificial intelligence tools use this simple principle called Markov chain in some form.
- Markov chain is based on a principle of "memorylessness". In other words, it is a stochastic model that describes the probabilities of transition to the next state of the process based solely on the current state of the system and not the series of states.

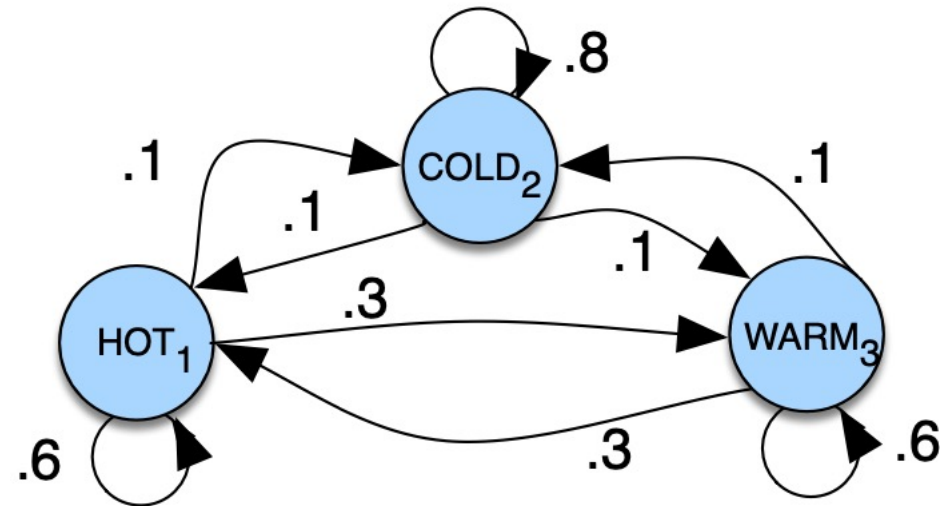
Markov Chains, why?

- Markov chains are used to analyze trends and predict the future (weather, stock market, genetics, product success, etc.

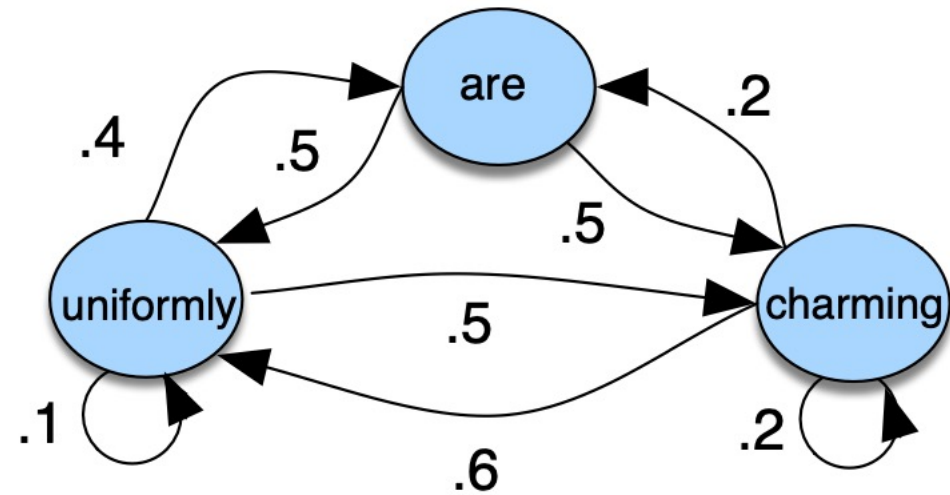


Markov chains

To predict tomorrow's weather you could examine today's weather but you weren't allowed to look at yesterday's weather



(a)



(b)

A Markov chain for weather (a) and one for words (b), showing states and transitions. A start distribution π is required; setting $\pi = [0.1, 0.7, 0.2]$ for (a) would mean a probability 0.7 of starting in state 2 (cold), probability 0.1 of starting in state 1 (hot), etc.

Markov assumption

Consider a sequence of state variables $q_1, q_2, \dots, q_i$. A Markov model embodies the Markov assumption on the probabilities of this sequence: that when predicting the future, the past doesn't matter, only the present.

Markov Assumption: $P(q_i = a | q_1 \dots q_{i-1}) = P(q_i = a | q_{i-1})$

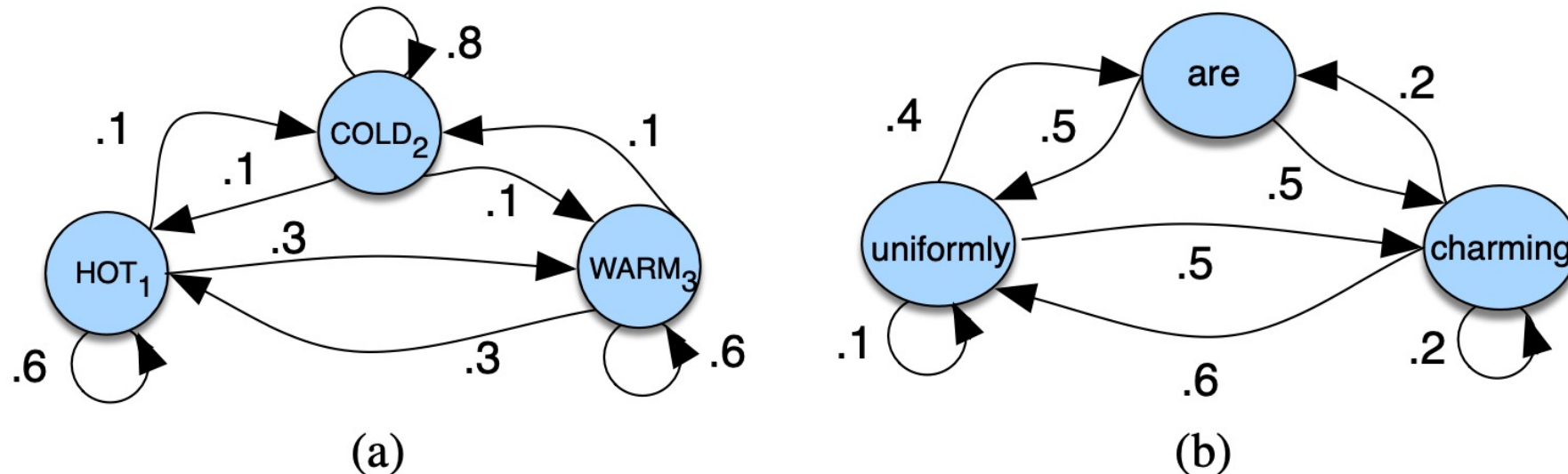


Figure a shows a Markov chain for assigning a probability to a sequence of weather events, for which the vocabulary consists of HOT, COLD, and WARM. Figure b shows a Markov chain for assigning a probability to a sequence of words $w_1 \dots w_t$

Markov chain components

Formally, a Markov chain is specified by the following components

$$Q = q_1 q_2 \dots q_N$$

a set of N states

$$A = a_{11} a_{12} \dots a_{N1} \dots a_{NN}$$

a **transition probability matrix** A , each a_{ij} representing the probability of moving from state i to state j , s.t.
 $\sum_{j=1}^n a_{ij} = 1 \quad \forall i$

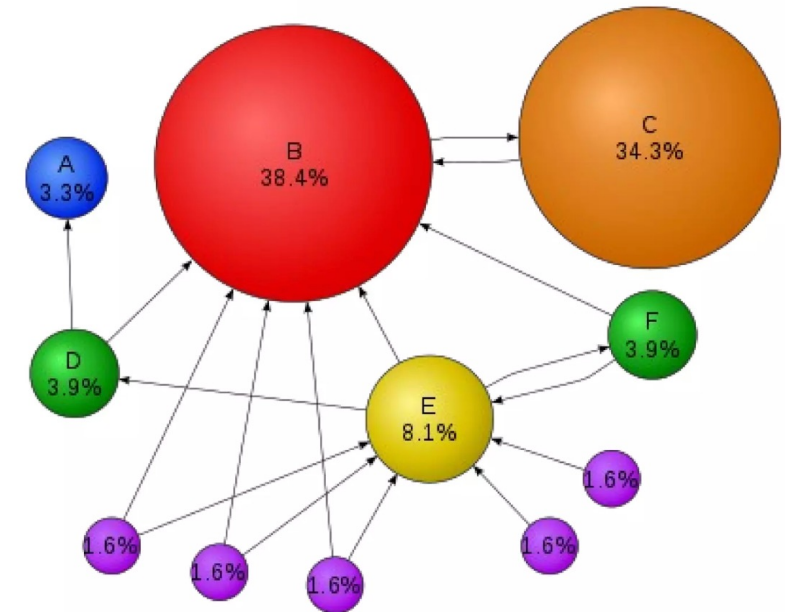
$$\pi = \pi_1, \pi_2, \dots, \pi_N$$

an **initial probability distribution** over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^n \pi_i = 1$

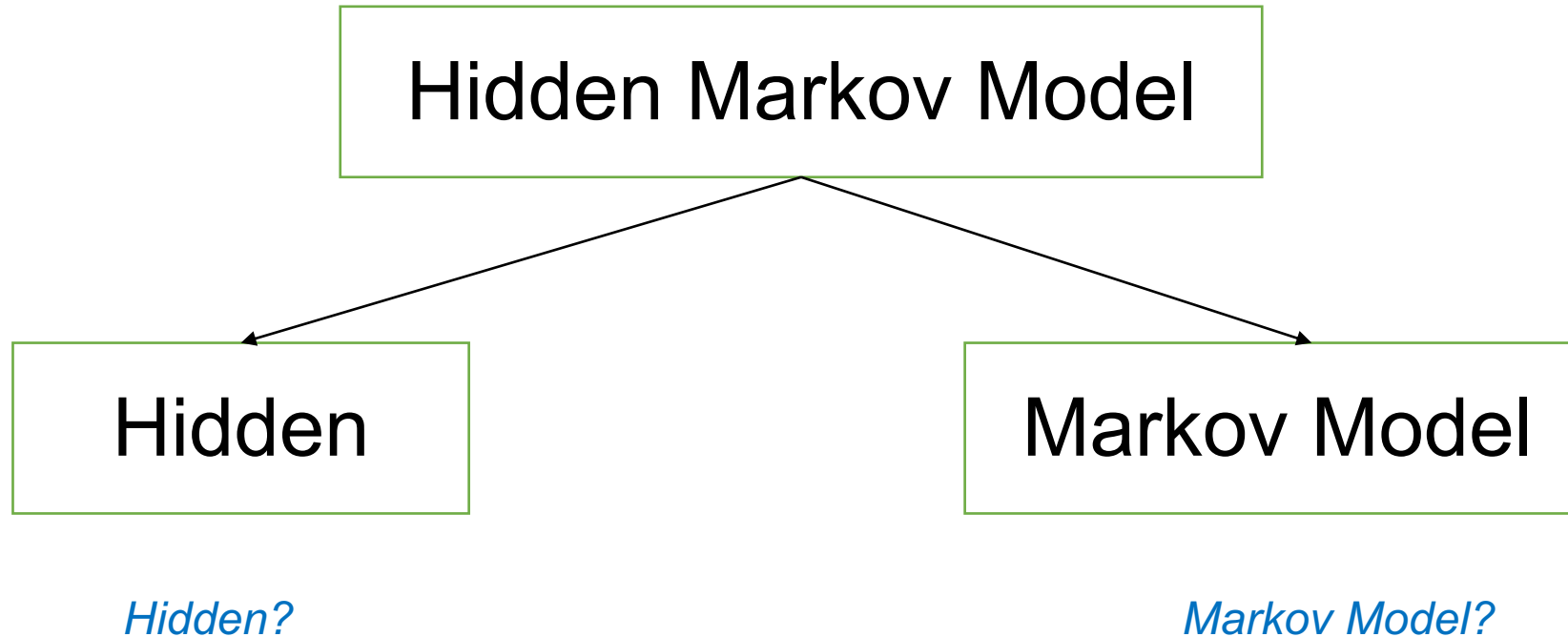
Markov chains applications

- Market research problems (Market share)
- Markov text generators
- Asset pricing and other financial predictions
- Customer journey predictions
- Population genetics
- Algorithmic music composition
- Page ranks (ex: Google results)

Page Rank

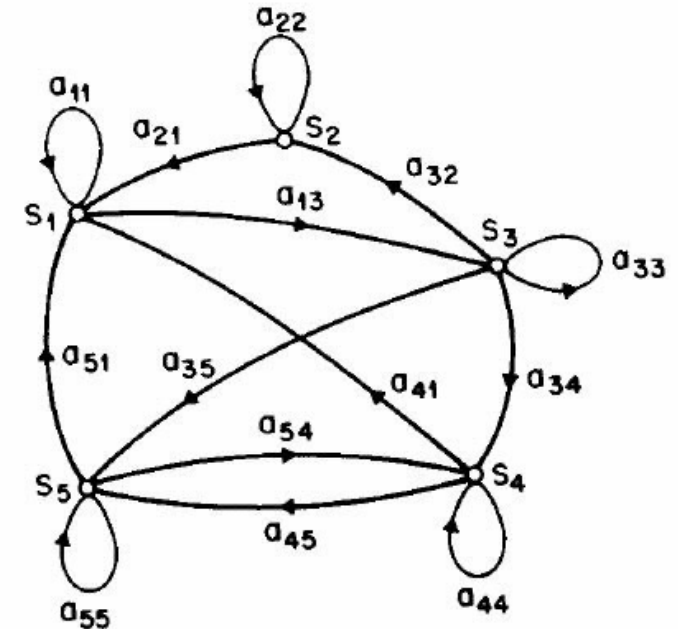


Hidden Markov Model



Markov Model

Markov Model is a stochastic model used to model randomly changing systems where it is assumed that future states depend only on the present state and not on the sequence of events that preceded it



A Markov chain with 5 states (S1 to S5) with selected state transitions

Hidden Markov Model

- Probabilistic generative model for sequences.
- Assume an underlying set of *hidden* (unobserved, latent) states in which the model can be (e.g. parts of speech).
- Assume probabilistic transitions between states over time (e.g. transition from POS to another POS as sequence is generated).
- Assume a *probabilistic* generation of tokens from states (e.g. words generated for each POS).

Hidden Markov Model Components

A hidden Markov model (HMM) allows us to talk about both *observed* events (like words that we see in the input) and *hidden* events (like part-of-speech tags) that we think of as causal factors in our probabilistic model

$Q = q_1 q_2 \dots q_N$	a set of N states
$A = a_{11} \dots a_{ij} \dots a_{NN}$	a transition probability matrix A , each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^N a_{ij} = 1 \quad \forall i$
$O = o_1 o_2 \dots o_T$	a sequence of T observations , each one drawn from a vocabulary $V = v_1, v_2, \dots, v_V$
$B = b_i(o_t)$	a sequence of observation likelihoods , also called emission probabilities , each expressing the probability of an observation o_t being generated from a state q_i
$\pi = \pi_1, \pi_2, \dots, \pi_N$	an initial probability distribution over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^n \pi_i = 1$

The Components Of An HMM Tagger

An HMM has two components, the A and B probabilities.

- The **A matrix** contains the tag transition probabilities $P(t_i|t_{i-1})$ which represent the probability of a tag occurring given the previous tag.
- The **B emission probabilities**, $P(w_i|t_i)$, represent the probability, given a tag, that it will be associated with a given word (say *will*)

We compute the maximum likelihood estimate of this transition probability

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

The Components Of An HMM Tagger

- In the WSJ corpus, for example, MD (modal auxiliary verb) occurs 13124 times of which it is followed by verb base (VB) 10471, for an MLE estimate of:

$$P(VB|MD) = \frac{C(MD, VB)}{C(MD)} = \frac{10471}{13124} = .80$$

- The MLE of the emission probability is:

$$P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

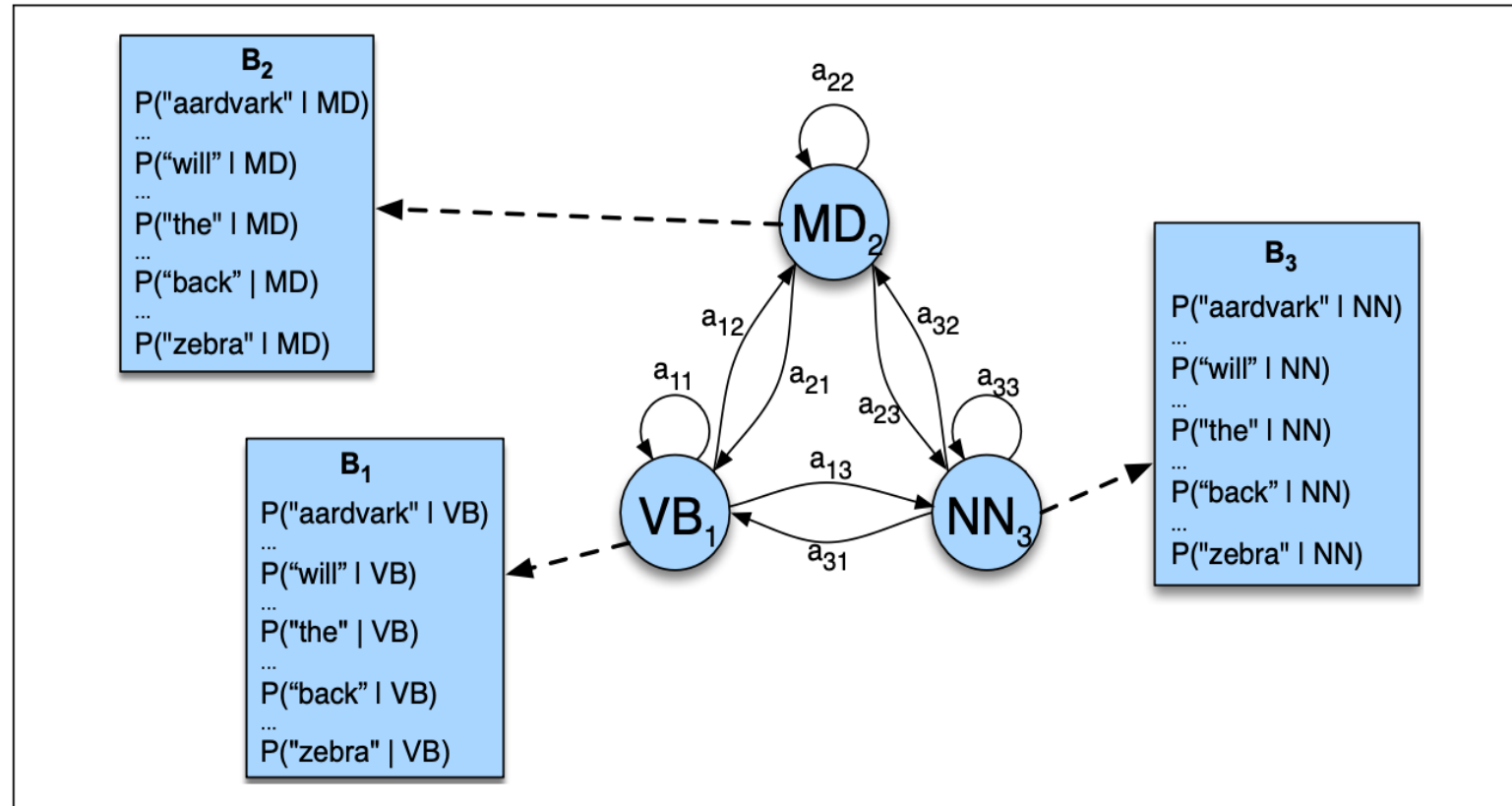
- Of the 13124 occurrences of MD in the WSJ corpus, it is associated with *will* 4046 times

$$P(will|MD) = \frac{C(MD, will)}{C(MD)} = \frac{4046}{13124} = .31$$

The Components Of An HMM Tagger

The A transition probabilities, and B observation likelihoods of the HMM are illustrated in figure for three states in an HMM part-of-speech tagger.

- the **A transition probabilities** used to compute the prior probability
- the **B observation likelihoods** that are associated with each state, one likelihood for each possible observation word



HMM Tagging As Decoding

- The task of determining the hidden variables sequence corresponding to the sequence of observations is called **decoding**.
- Decoding given as input an HMM $\lambda = (A, B)$ and a sequence of observations $O = o_1, o_2, \dots, o_T$, find the most probable sequence of states $Q = q_1 q_2 q_3 \dots q_T$.
- For POS tagging, **the goal** of HMM decoding is **to choose the tag sequence** $t_1 \dots t_n$ that is most probable given the observation sequence of n words $w_1, \dots, w_n$

$$\hat{t}_{1:n} = \operatorname{argmax}_{t_1 \dots t_n} P(t_1 \dots t_n | w_1 \dots w_n) \quad (1)$$

HMM Tagging As Decoding

HMM taggers make two further simplifying assumptions. The first is that the probability of a word appearing depends only on its own tag and is independent of neighboring words and tags:

$$P(w_1 \dots w_n | t_1 \dots t_n) \approx \prod_{i=1}^n P(w_i | t_i) \quad (2)$$

The second assumption, the bigram assumption, is that the probability of a tag is $P(w_i | t_i)$ (1) dependent only on the previous tag, rather than the entire tag sequence

$$P(t_1 \dots t_n) \approx \prod_{i=1}^n P(t_i | t_{i-1}) \quad (3)$$

Plugging the simplifying assumptions from Eq. (2) and Eq. (3) into Eq. (1) results in the following equation for the most probable tag sequence from a bigram tagger

$$\hat{t}_{1:n} = \operatorname{argmax}_{t_1 \dots t_n} P(t_1 \dots t_n | w_1 \dots w_n) \approx \operatorname{argmax}_{t_1 \dots t_n} \prod_{i=1}^n \overbrace{P(w_i | t_i)}^{\text{emission}} \overbrace{P(t_i | t_{i-1})}^{\text{transition}}$$

The Viterbi Algorithm

The decoding algorithm for HMMs is the Viterbi algorithm, as shown below.

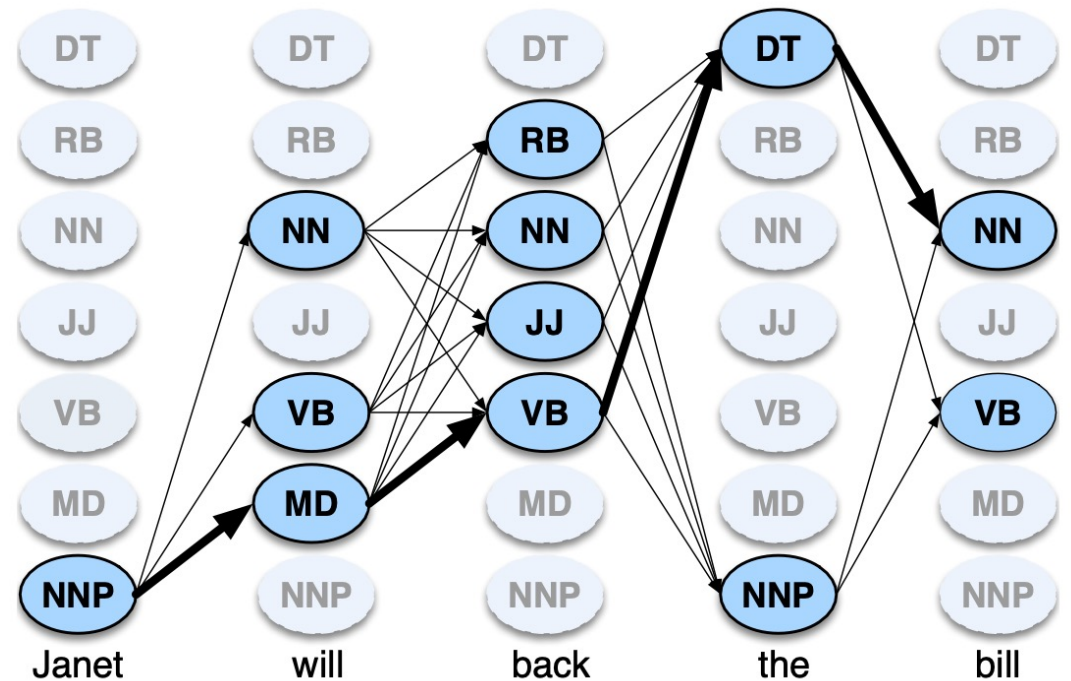
```
function VITERBI(observations of len  $T$ , state-graph of len  $N$ ) returns best-path, path-prob

create a path probability matrix viterbi[ $N, T$ ]
for each state  $s$  from 1 to  $N$  do                                ; initialization step
    viterbi[ $s, 1$ ]  $\leftarrow \pi_s * b_s(o_1)$ 
    backpointer[ $s, 1$ ]  $\leftarrow 0$ 
for each time step  $t$  from 2 to  $T$  do                            ; recursion step
    for each state  $s$  from 1 to  $N$  do
        viterbi[ $s, t$ ]  $\leftarrow \max_{s'=1}^N \text{viterbi}[s', t-1] * a_{s',s} * b_s(o_t)$ 
        backpointer[ $s, t$ ]  $\leftarrow \operatorname{argmax}_{s'=1}^N \text{viterbi}[s', t-1] * a_{s',s} * b_s(o_t)$ 

bestpathprob  $\leftarrow \max_{s=1}^N \text{viterbi}[s, T]$                         ; termination step
bestpathpointer  $\leftarrow \operatorname{argmax}_{s=1}^N \text{viterbi}[s, T]$             ; termination step
bestpath  $\leftarrow$  the path starting at state bestpathpointer, that follows backpointer[] to states back in time
return bestpath, bestpathprob
```

The Viterbi Algorithm

- A sketch of the lattice for *Janet will back the bill*, showing the possible tags (q_i) for each word and highlighting the path corresponding to the correct tag sequence through the hidden states.
- States (parts of speech) which have a zero probability of generating a particular word according to the B matrix (such as the probability that a determiner DT will be realized as *Janet*) are greyed out.



Example

Janet will back the bill

The goal is the correct series of tags

Janet/NNP will/MD back/VB the/DT bill/NN

	NNP	MD	VB	JJ	NN	RB	DT
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

The A transition probabilities $P(t_i | t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(VB | MD)$ is 0.7968.

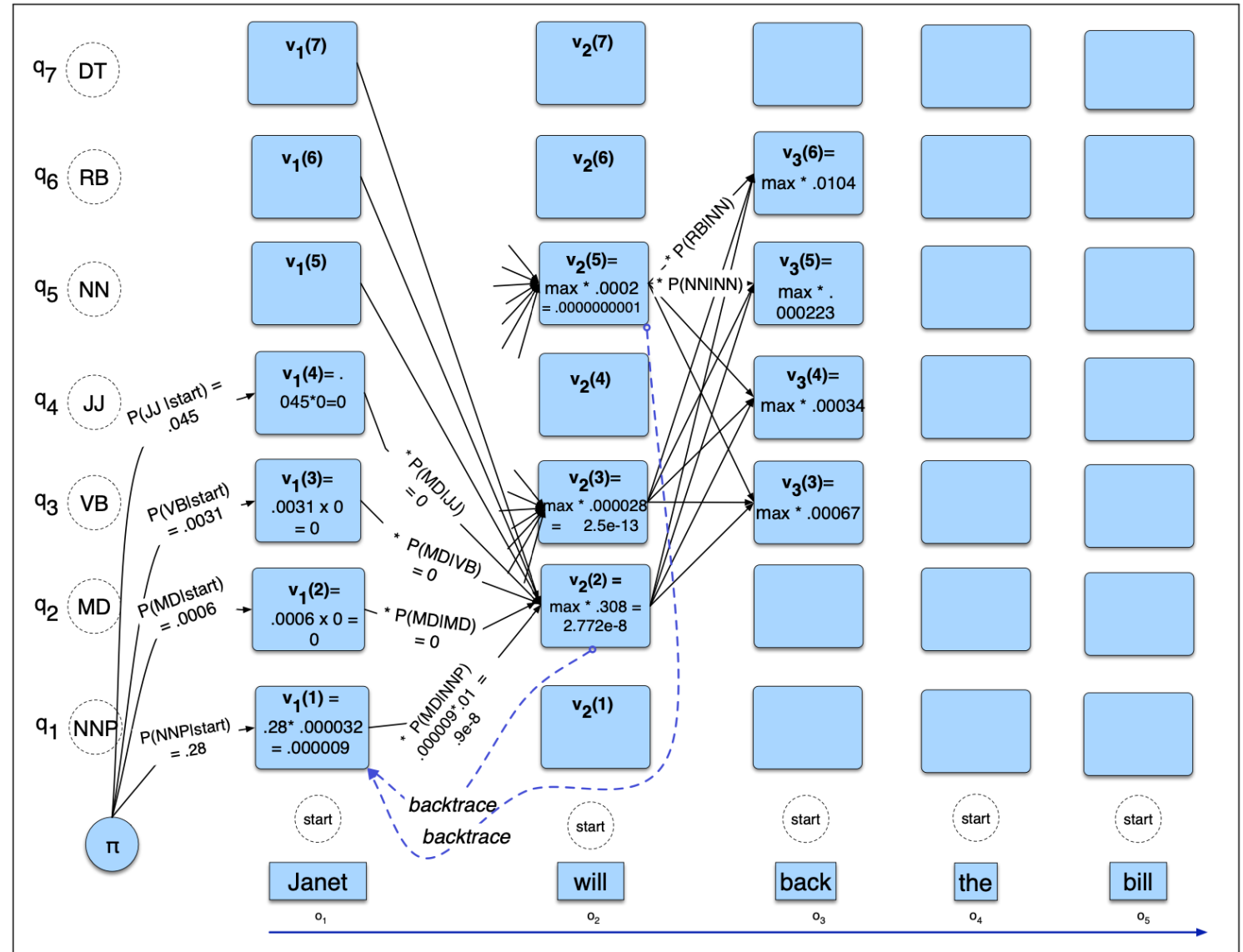
Example

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Observation likelihoods B computed from the WSJ corpus without smoothing, simplified slightly.

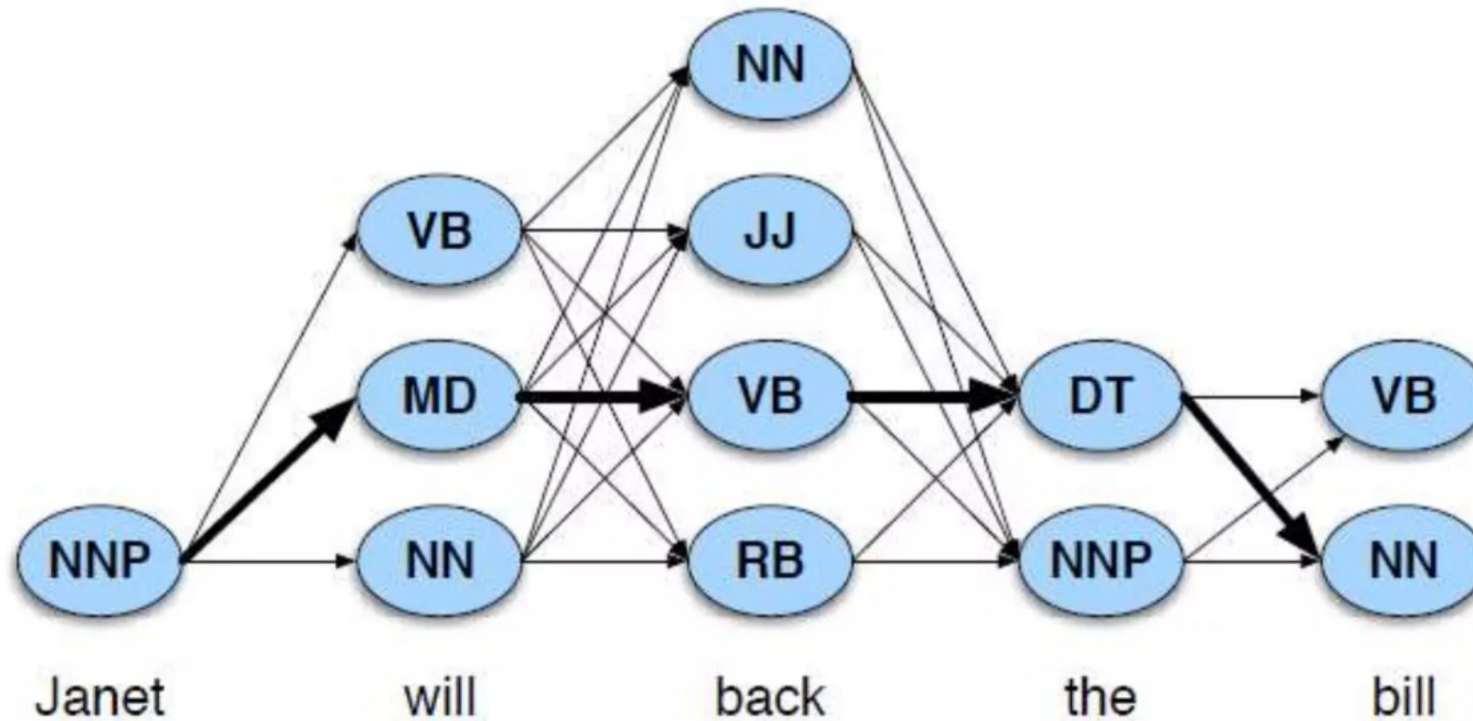
Example

- ✓ The first few entries in the individual state columns for the Viterbi algorithm.
- ✓ Each cell keeps the probability of the best path so far and a pointer to the previous cell along that path.
- ✓ We have only filled out columns 1 and 2; to avoid clutter most cells with value 0 are left empty. The rest is left as an exercise for the reader.
- ✓ After the cells are filled in, backtracing from the *end* state, we should be able to reconstruct the correct state sequence NNP MD VB DT NN



Example

Following figure shows a schematic of the possible tags for each word and the correct final tag sequence



Decoding: Beam Search

- Generate tags for w_1 , find top N , set s_{1j} accordingly, $j=1, 2, \dots, N$
- For $i=2$ to n (n is the sentence length)
 - For $j=1$ to N
 - Generate tags for w_i , given $S_{(i-1)j}$ as previous tag context
 - Append each tag to $S_{(i-1)j}$ to make a new sequence.
 - Find N highest prob sequences generated above, and set s_{ij} accordingly, $j=1, \dots, N$
- Return highest prob sequence S_{n1}

The Beam Search algorithm for POS tagging helps search through choices for words in the sentence and construct potential tag sequences based on probabilities, then select the sequence with the highest probability as the final result.

Beam Search

- Beam interence:
 - At each position keep the top k complete sequences.
 - Extend each sequence in each local way.
 - The extensions compete for the k slots at the next position.Advantages:
- Advantages:
 - Fast; and beam sizes of 3-5 are as good or almost as good as exact inference in many cases.
 - Easy to implement (no dynamic programming required).
- Disadvantage:
 - Inexact: the globally best sequence can fall off the beam.

Natural Language Processing

Chapter 3: Statistical Language Models

PhD. Nguyen Thi Mai Trang



Content

N-gram language models

- Language models
- N-gram language models

Evaluating language models

- Extrinsic and intrinsic evaluation
- Perplexity

Generalization

Smoothing

- Laplace smoothing
- Add-k smoothing
- Backoff and interpolation

N-gram Language Models

- Formal grammars (e.g. regular, context free) give a hard “binary” model of the legal sentences in a language.
- For NLP, a *probabilistic* model of a language that gives a probability that a string is a member of a language is more useful.
- To specify a correct probability distribution, the probability of all sentences in a language must sum to 1.
- Models that assign probabilities to sequences of words are called language models or **LMs**.
- An n-gram is a sequence of n words:
 - a 2-gram (a bigram) is a two-word sequence of words like “please turn”, “turn your”, or “your homework”,
 - 3-gram (a trigram) is a three-word sequence of words like “please turn your”, or “turn your homework”

N-gram Models

- Estimate probability of each word given prior context.
 - $P(\text{phone} \mid \text{Please turn off your cell})$
- Number of parameters required grows exponentially with the number of words of prior context.
- An N-gram model uses only $N-1$ words of prior context.
 - Unigram: $P(\text{phone})$
 - Bigram: $P(\text{phone} \mid \text{cell})$
 - Trigram: $P(\text{phone} \mid \text{your cell})$
- The *Markov assumption* is the presumption that the future behavior of a dynamical system only depends on its recent history. In particular, in a *k th-order Markov model*, the next state only depends on the k most recent states, therefore an N-gram model is a $(N-1)$ -order Markov model.

N-gram Model Formulas

- Word sequences

$$w_1^n = w_1 \dots w_n$$

- Chain rule of probability

$$P(w_1^n) = P(w_1)P(w_2 | w_1)P(w_3 | w_1^2) \dots P(w_n | w_1^{n-1}) = \prod_{k=1}^n P(w_k | w_1^{k-1})$$

- Bigram approximation

$$P(w_1^n) = \prod_{k=1}^n P(w_k | w_{k-1})$$

- N-gram approximation

$$P(w_1^n) = \prod_{k=1}^n P(w_k | w_{k-N+1}^{k-1})$$

Estimating Probabilities

N-gram conditional probabilities can be estimated from raw text based on the *relative frequency* of word sequences.

$$\textbf{Bigram:} \quad P(w_n \mid w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

$$\textbf{N-gram:} \quad P(w_n \mid w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}w_n)}{C(w_{n-N+1}^{n-1})}$$

To have a consistent probabilistic model, append a unique start (<s>) and end (</s>) symbol to every sentence and treat these as additional words.

Generative Model & MLE

An N-gram model can be seen as a probabilistic automata for generating sentences.

Initialize sentence with N-1 <s> symbols

Until </s> is generated do:

Stochastically pick the next word based on the conditional probability of each word given the previous N - 1 words.

Relative frequency estimates can be proven to be *maximum likelihood estimates* (MLE) since they maximize the probability that the model M will generate the training corpus T .

$$\hat{\lambda} = \operatorname{argmax}_{\lambda} P(T \mid M(\lambda))$$

Example

$$\begin{aligned} &P(<s> \text{ i want english food } </s>) \\ &= P(\text{i} \mid <s>) P(\text{want} \mid \text{i}) P(\text{english} \mid \text{want}) \\ &\quad P(\text{food} \mid \text{english}) P(</s> \mid \text{food}) \\ &= 0.25 \times 0.33 \times 0.0011 \times 0.5 \times 0.68 = 0.000031 \end{aligned}$$

$$\begin{aligned} &P(<s> \text{ i want chinese food } </s>) \\ &= P(\text{i} \mid <s>) P(\text{want} \mid \text{i}) P(\text{chinese} \mid \text{want}) \\ &\quad P(\text{food} \mid \text{chinese}) P(</s> \mid \text{food}) \\ &= 0.25 \times 0.33 \times 0.0065 \times 0.52 \times 0.68 = 0.00019 \end{aligned}$$

Evaluating Language Models

- Ideally, evaluate use of model in end application (*extrinsic, in vivo*)
 - Realistic
 - Expensive
- Evaluate on ability to model test corpus (*intrinsic*).
 - Less realistic
 - Cheaper
- Verify at least once that intrinsic evaluation correlates with an extrinsic one.

Perplexity

- Measure of how well a model “fits” the test data.
- Uses the probability that the model assigns to the test corpus.
- Normalizes for the number of words in the test corpus and takes the inverse.

$$PP(W) = \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}}$$

- Measures the weighted average branching factor in predicting the next word (lower is better).

Perplexity Evaluation

- Models trained on 38 million words from the Wall Street Journal (WSJ) using a 19,979 word vocabulary.
- Evaluate on a disjoint set of 1.5 million WSJ words.

	Unigram	Bigram	Trigram
Perplexity	962	170	109

Generalization

- Generalization in Machine Learning
 - The ability of a model to perform well on new, unseen data.
 - A key measure of a model's effectiveness.
- Factors Affecting Generalization
 - Data quality and quantity.
 - Model complexity and regularization.
 - The impact of hyperparameters.

Achieving Optimal Generalization

- Balancing Bias and Variance
 - Bias: Underfitting occurs when the model is too simple and doesn't capture the data's complexity.
 - Variance: Overfitting occurs when the model is too complex and fits the noise in the data.
- Leveraging Cross-Validation
 - Divide data into training, validation, and test sets.
 - Assess model performance on validation data to fine-tune hyperparameters.
- Consider Transfer Learning
 - Utilize pre-trained models for new tasks.
 - Adapt models with less data, saving training time and resources.

Smoothing

- Since there are a combinatorial number of possible word sequences, many rare (but not impossible) combinations never occur in training, so MLE incorrectly assigns zero to many parameters (a.k.a. *sparse data*).
- If a new combination occurs during testing, it is given a probability of zero and the entire sequence gets a probability of zero (i.e. infinite perplexity).
- In practice, parameters are *smoothed* (a.k.a. *regularized*) to reassign some probability mass to unseen events.
 - Adding probability mass to unseen events requires removing it from seen ones (*discounting*) in order to maintain a joint distribution that sums to 1.

Laplace (Add-One) Smoothing

- “Hallucinate” additional training data in which each possible N-gram occurs exactly once and adjust estimates accordingly.

$$\textbf{Bigram:} \quad P(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V}$$

$$\textbf{N-gram:} \quad P(w_n | w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}w_n) + 1}{C(w_{n-N+1}^{n-1}) + V}$$

where V is the total number of possible $(N-1)$ -grams (i.e. the vocabulary size for a bigram model).

- Tends to reassign too much mass to unseen events, so can be adjusted to add $0 < \delta < 1$ (normalized by δV instead of V).

Add-k Smoothing

- One alternative to add-one smoothing is to move a bit less of the probability mass from the seen to the unseen events
- Instead of adding 1 to each count, we add a fractional count k (.5? .05? .01?)

$$P_{\text{Add-k}}^*(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n) + k}{C(w_{n-1}) + kV}$$

- Add-k smoothing is a valuable technique.
- We need a method for selecting the optimal value of k . This can be achieved by optimizing on a development set.
- Add-k is effective in tasks like text classification. However, it faces limitations in language modeling. It can result in counts with inadequate variances.

Advanced Smoothing

- Many advanced techniques have been developed to improve smoothing for language models.
 - Good-Turing
 - Interpolation
 - Backoff
 - Kneser-Ney
 - Class-based (cluster) N-grams

Model Combination

- As N increases, the power (expressiveness) of an N -gram model increases, *but* the ability to estimate accurate parameters from sparse data decreases (i.e. the smoothing problem gets worse).
- A general approach is to combine the results of multiple N -gram models of increasing complexity (i.e. increasing N).

Interpolation

- Linearly combine estimates of N-gram models of increasing order.
- Interpolated Trigram Model:

$$\hat{P}(w_n | w_{n-2}, w_{n-1}) = \lambda_1 P(w_n | w_{n-2}, w_{n-1}) + \lambda_2 P(w_n | w_{n-1}) + \lambda_3 P(w_n)$$

$$\text{Where: } \sum_i \lambda_i = 1$$

- Learn proper values for λ_i by training to (approximately) maximize the likelihood of an independent *development* (a.k.a. *tuning*) corpus.

Backoff

- Only use lower-order model when data for higher-order model is unavailable (i.e. count is zero).
- Recursively back-off to weaker models until data is available.

$$P_{katz}(w_n | w_{n-N+1}^{n-1}) = \begin{cases} P^*(w_n | w_{n-N+1}^{n-1}) & \text{if } C(w_{n-N+1}^n) > 1 \\ \alpha(w_{n-N+1}^{n-1}) P_{katz}(w_n | w_{n-N+2}^{n-1}) & \text{otherwise} \end{cases}$$

Where P^* is a discounted probability estimate to reserve mass for unseen events and α 's are back-off weights (see text for details).

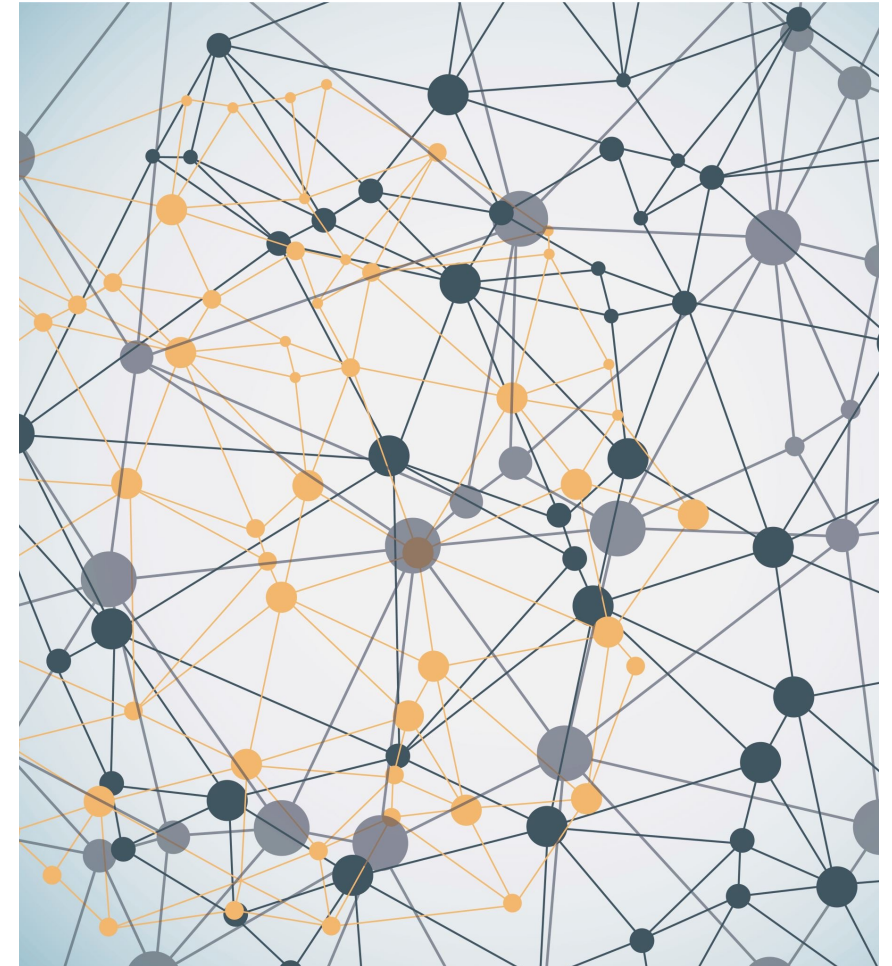
A Problem for N-Grams: Long Distance Dependencies

- Many times local context does not provide the most useful predictive clues, which instead are provided by *long-distance dependencies*.
 - Syntactic dependencies
 - “The *man* next to the large oak tree near the grocery store on the corner **is** tall.”
 - “The *men* next to the large oak tree near the grocery store on the corner **are** tall.”
 - Semantic dependencies
 - “The *bird* next to the large oak tree near the grocery store on the corner **flies** rapidly.”
 - “The *man* next to the large oak tree near the grocery store on the corner **talks** rapidly.”
- More complex models of language are needed to handle such dependencies.

Natural Language Processing

Chapter 4: Sentiment Classification

PhD. Nguyen Thi Mai Trang



Content

- Classification
 - Supervised machine learning
 - Classification
- Naïve Bayes classifiers
 - The method
 - Training
 - Worked example
 - Sentiment classification with Naïve Bayes
- Evaluation
 - Precision, Recall, F-measure
 - Test sets and Cross-validation
- Projects

Opinion Mining - Sentiment Classification

For a given Text Document, Determine Sentiment Orientation

- Positive or Negative, Favorable or Unfavorable, etc.
- Binary or along a scale (e.g. 1-5 stars)
- Data is unstructured text format. From sentence to document level.

Example: *Positive* or *Negative*?

"This is by far the worst hotel experience i've ever had. The owner overbooked while i was staying there (even though i booked the room two months in advance) and made me move to another room, but that room wasn't even a hotel room!"

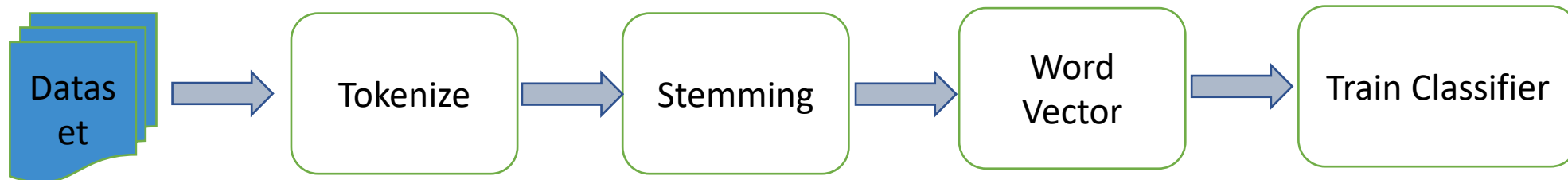
Supervised Learning For Text

- Train a classifier algorithm based on a training data set.
 - Raw data will be text.
- Approach: Use term presence information as features
 - A plain text document becomes a word vector

ID	Ant	Book	Car	Food	...
1	0	1	0	0	
2	1	0	0	0	
3	0	0	0	1	

Supervised Learning For Text

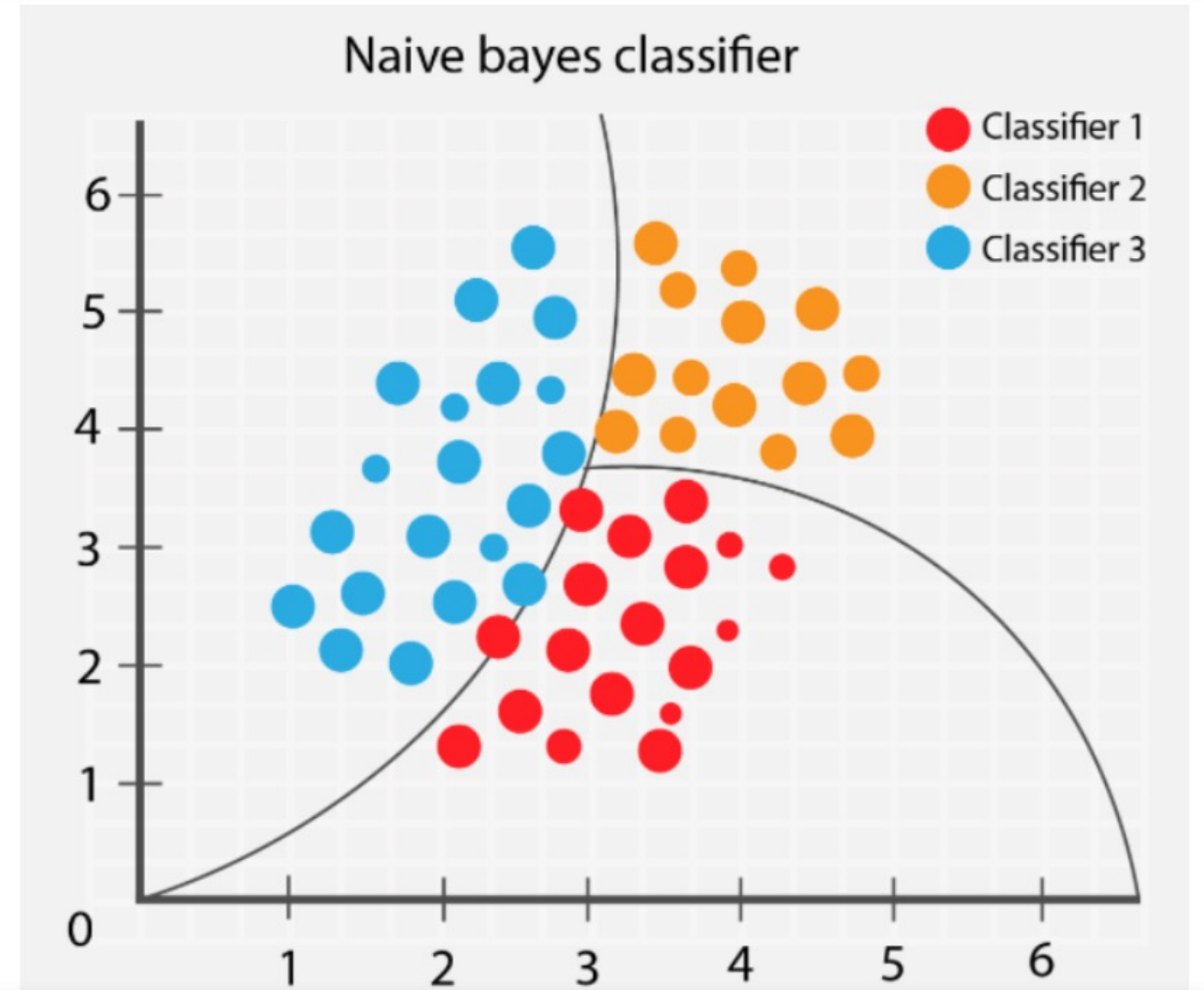
- A word vector can be used to train a classifier
- Building a Word Vector
 - Unit of tokenization: uni/bi/n-gram
 - Term presence metric
 - Binary, tf-idf, frequency
 - Stemming
 - Stop words removal



Opinion Mining - Sentiment Classification

- Challenges of Data Driven Approaches
- Domain dependence
- Lose discourse information
- NLP techniques might help

Naïve Bayes classifier



Naive Bayes Classifier

Definition

In machine learning, Naive Bayes classifiers are a family of simple probabilistic classifiers based on applying Bayes's theorem with strong (naive) independence assumptions between the features.

Potential Use Cases

It is one of the most basic text classification techniques with various applications

- Email Spam Detection
- Language Detection
- Sentiment Detection
- Personal email sorting
- Document categorization

Advantages

- Only utilize the simple probability and Bayes' theorem
- Computational efficiency

Bayes Theorem

Given a hypothesis h and data D which bears on the hypothesis:

$$P(h | D) = \frac{P(D).P(H)}{P(D)}$$

$P(h)$: independent probability of h : prior probability

$P(D)$: independent probability of D

$P(D | h)$: conditional probability of D given h : likelihood

$P(h | D)$: conditional probability of h given D : posterior probability

Maximum A Posterior

- Based on Bayes Theorem, we can compute the Maximum A Posterior (MAP) hypothesis for the data
- The best hypothesis for some space H given observed training data D is being sought

$$\begin{aligned}h_{MAP} &\equiv \operatorname{argmax}_{h \in H} P(h|D) \\&\equiv \operatorname{argmax}_{h \in H} \frac{P(D|h)P(h)}{P(D)} \\&\equiv \operatorname{argmax}_{h \in H} P(D|h)P(h)\end{aligned}$$

H - set of all hypothesis

Not that we can drop $P(D)$ as the probability of the data is constant (and independent of the hypothesis)

Maximum Likelihood

- Now assume that all hypothesis H are equally probable a prior, i.e. $P(h_i) = P(h_j)$ for all h_i, h_j belong to H .
- This is called assuming a uniform prior. It simplifies computing the posterior:

$$h_{ML} = \operatorname{argmax}_{h \in H} P(D|h)$$

- This hypothesis is called the maximum likelihood hypothesis

Bayesian Classifier

D: set of tuples

Each tuple is an 'n' dimensional attribute vector $X (x_1, x_2, x_3, \dots, x_n)$

Where x_i is the value of attribute A_i

Let there are 'm' classes: $C_1, C_2, C_3, \dots, C_m$

Bayesian classifier predicts X belongs to Class C_i if

$P(C_i|X) > P(C_j|X)$ for $1 \leq j \leq m, j \neq i$

Naive Bayesian Classifier

- With many attributes, it is computationally expensive to evaluate $P(X | C_i)$
- Naive assumption of “class conditional independence”

$$P(X | c_i) = P(x_1, x_2, \dots, x_n | C_i)$$

$$= P(x_1 | C_i) * P(x_2 | C_i) * \dots * P(x_n | C_i)$$

$$= \prod_{k=1}^n P(x_k | C_i)$$

Naive Bayesian Classifier

- To compute $P(x_k|C_i)$
- A_k is a categorical:
 - $P(x_k|C_i) = \frac{\text{the number of tuples of class } C_i \text{ in } D \text{ having the value } x_k \text{ for } A_k}{\text{the number of tuples of class } C_i \text{ in } D}$
- A_k is continuous:
- A continuous-valued attribute is typically assumed to have a Gaussian distribution with a mean μ and standard deviation σ

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}};$$

$$P(x_k|C_i) = g(x_k, \mu_i, \sigma_{C_i})$$

Zero Problem

What if there is a class, C_i and X has an attribute value, x_k , such that none of the samples in C_i has that attribute value?

⇒ In that case $P(x_k | c_i) = 0$, which results in $P(X | C_i) = 0$ even though $P(x_k | C_i)$ for all the other attributes in X may be large.

Basic Assumption

The Naive Bayes assumption is that all the features are conditionally independent given the class label.

$$\begin{aligned} P(X | C_i) &= P(x_1, x_2, \dots, x_n | C_i) \\ &= P(x_1 | C_i) * P(x_2 | C_i) * \dots * P(x_n | C_i) \end{aligned}$$

Even though this is usually false (since features are usually dependent)

Training The Model

To train a Naïve Bayes classifier, we first need to estimate **the prior probabilities** of each class and **the conditional probabilities** of each feature given each class. This can be done using maximum likelihood estimation or Bayesian estimation.

Example: Class-labeled Training From Database

Suppose we have a dataset of emails labeled as spam or not spam, and we want to classify a new email as either spam or not spam.

We can use a Naïve Bayes classifier to do this by first estimating the prior probabilities of spam and not spam emails, and then estimating the conditional probabilities of each word occurring in spam and not spam emails.

Email	Spam	Contains "money"
E1	Yes	No
E2	No	No
E3	Yes	Yes
E4	No	Yes
E5	Yes	Yes
E6	No	Yes

Example: Class-labeled Training From Database

Estimate the prior probabilities as follows:

- $P(\text{Spam}) = 3/6 = 0.5$
- $P(\text{Not Spam}) = 3/6 = 0.5$

Estimate the conditional probabilities as follows:

- $P(\text{Money} | \text{Spam}) = 1/3 = 0.33$
- $P(\text{Money} | \text{Not Spam}) = 1/3 = 0.33$

Now suppose we want to classify a new email that contains the word “money”. We can use Bayes’ theorem to calculate the probability that this email is spam:

$$\begin{aligned} P(\text{Spam} | \text{“money”}) &= P(\text{“money”} | \text{Spam}) * P(\text{Spam}) \\ &= (0.33) * (0.5) \\ &= 0.165 \end{aligned}$$

$$\begin{aligned} P(\text{Not Spam} | \text{“money”}) &\propto P(\text{“money”} | \text{Not Spam}) * P(\text{Not Spam}) \\ &= (0.33) * (0.5) \\ &= 0.165 \end{aligned}$$

Therefore, we cannot classify this email as either spam or not spam with certainty.

Sentiment Classification With Naïve Bayes

What is sentiment classification?

- Sentiment classification is the process of identifying the sentiment expressed in a piece of text. It is a common task in natural language processing and is used in applications such as social media monitoring, customer feedback analysis, and market research.

Data Preprocessing

Why is data preprocessing important?

- Data preprocessing is an important step in sentiment classification because it helps to improve the accuracy of the classifier. Some common preprocessing techniques include removing stop words, stemming, and lemmatization.

Example

Suppose we have the following sentence: “I love this movie! It’s amazing.”

After preprocessing, the sentence would look like this: “love movie amazing”.

Example

- Suppose we have the training data

Text	Sentiment
I love this movie!	Positive
This movie is terrible	Negative
It was boring.	Negative
This movie is great!	Positive

Step 1: Data Preprocessing

Tokenize the sentences and remove punctuation:

- "I love this movie"
- "This movie is terrible"
- "It was boring"
- "This movie is great"

Example

Step 2: Prior Probabilities (Priors)

We can estimate the prior probabilities as follows:

- $P(\text{Positive}) = 2/4 = 0.5$
- $P(\text{Negative}) = 2/4 = 0.5$

Step 3: Conditional Probabilities (Likelihoods)

We can estimate the conditional probabilities as follows:

Positive	I	love	this	movie	is	great	Total words
	1	1	2	2	1	1	8

Negative	this	movie	is	terrible	it	was	boring	Total words
	1	1	1	1	1	1	1	7

Example

Setp 5: Classify the sentence “**This movie is boring.**”

Calculate the posterior probabilities for both classes (Positive and Negative) using the Naïve Bayes formula:

- $P(\text{Positive} \mid \text{"this movie is boring"})$
 $= P(\text{Positive}) * P(\text{"this"} \mid \text{Positive}) * P(\text{"movie"} \mid \text{Positive}) * P(\text{"is"} \mid \text{Positive}) * P(\text{"boring"} \mid \text{Positive})$
 $= \frac{1}{2} * \frac{2}{8} * \frac{2}{8} * \frac{1}{8} * 0 = 0$
- $P(\text{Negative} \mid \text{"the movie is boring"})$
 $= P(\text{Negative}) * P(\text{"this"} \mid \text{Negative}) * P(\text{"movie"} \mid \text{Negative}) * P(\text{"is"} \mid \text{Negative}) * P(\text{"boring"} \mid \text{Negative})$
 $= \frac{1}{2} * \frac{1}{7} * \frac{1}{7} * \frac{1}{7} * \frac{1}{7} = 0.0002$

Therefore, we classify this sentence as negative.

Evaluation

	Predicted: 0	Predicted: 1
Actual: 0	True Negatives (TN)	False Positives (FP)
Actual: 1	False Negatives (FN)	True Positives (TP)

Confusion matrix

Precision is a metric that shows the number of correct positive predictions. It is calculated as the number of true positives divided by the sum of true positives and false positives

$$\text{Precision} = \frac{TP}{TP + FP}$$

Evaluation

Recall is the most useful metric for many classification problems. It reports the number of correct predictions for the positive class made out of all positive class predictions.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

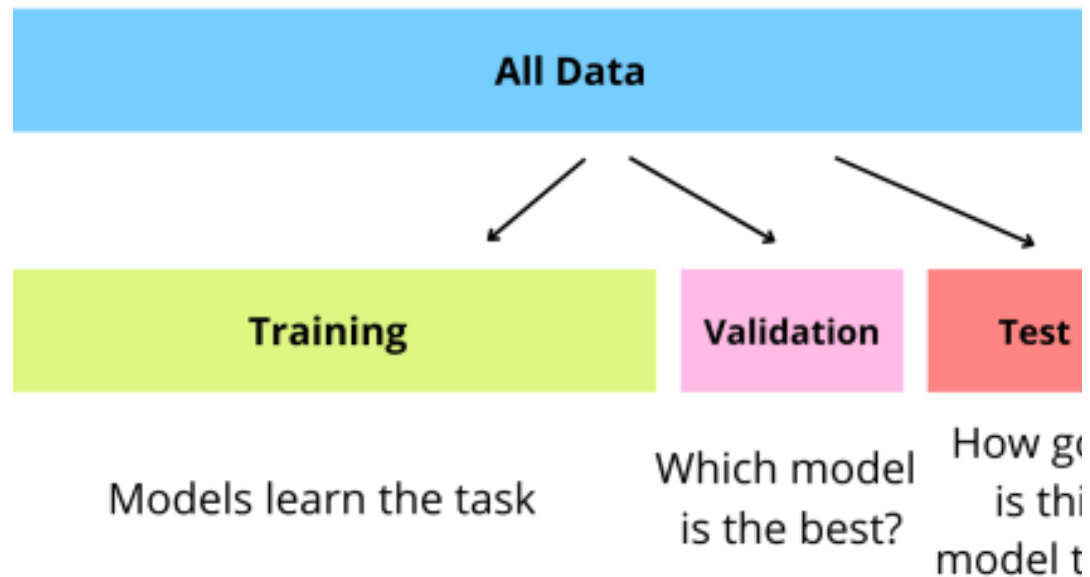
F-measures or F-score is calculated from the precision and recall of the test. A more general F score F_β , that uses a positive real factor β , where β is chosen such that recall is considered β times as important as precision, is:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{(\beta^2 \cdot \text{precision}) + \text{recall}}.$$

The traditional F-measure or balanced F-score (**F₁ score**) is the harmonic mean of precision and recall:

$$\text{F1 - score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

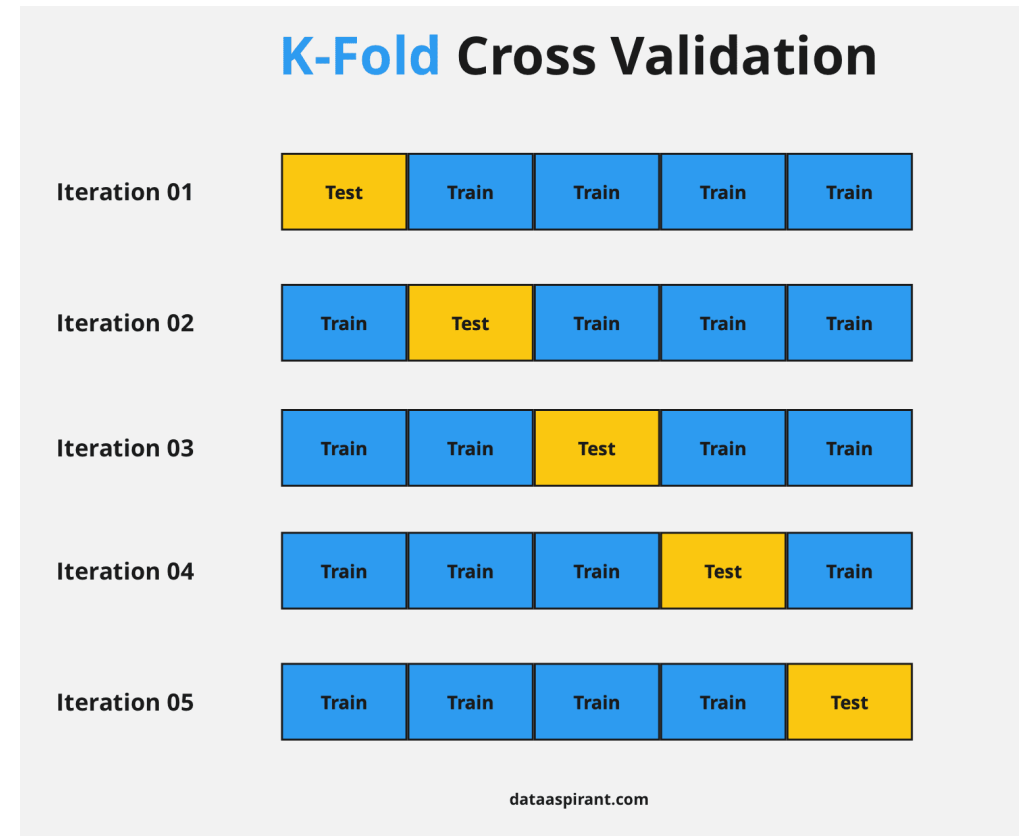
Importance of Test Sets



- **Test Sets:** Separate dataset used for evaluating the model's performance.
- **Unseen Data:** Test sets simulate how the model will perform on new, unseen data.
- **Prevent Overfitting:** Test sets help prevent overfitting and provide an unbiased evaluation.

Cross-validation

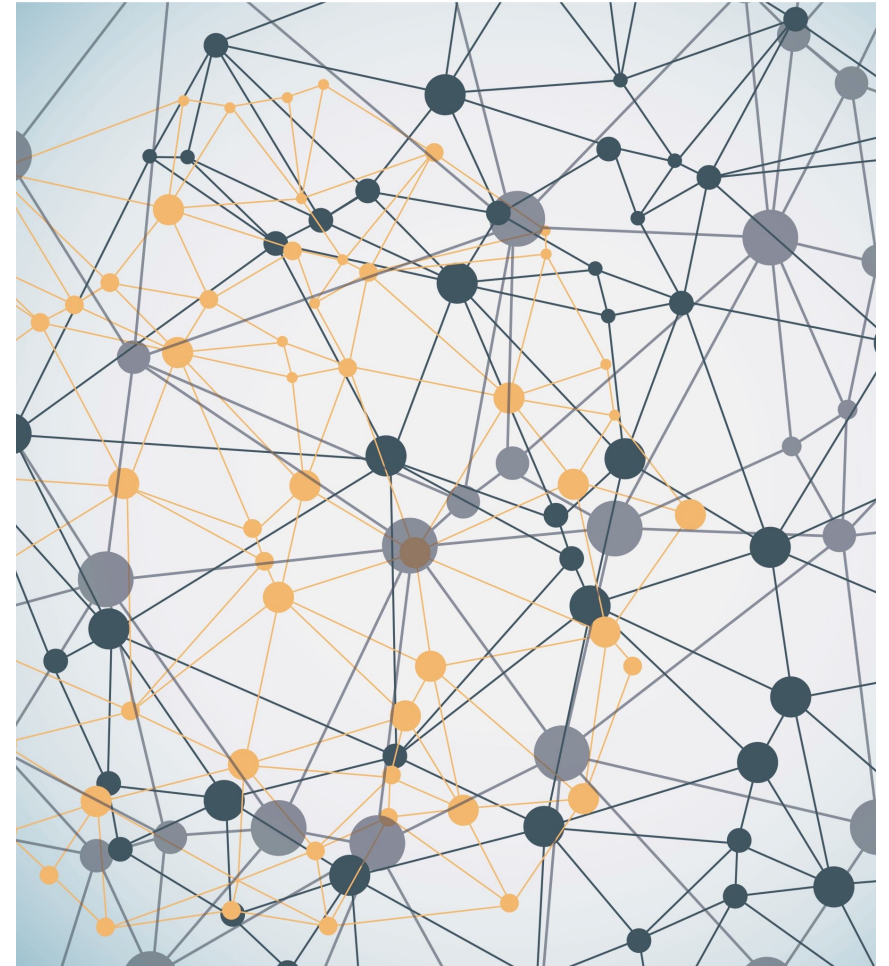
- Cross-validation: Technique to assess a model's performance more robustly.
- k-fold Cross-validation: Data is split into k subsets, and the model is trained and tested k times.
- Average Metrics: Compute average precision, recall, and F-measure to evaluate the model comprehensively.



Natural Language Processing

Chapter 5: Syntactic Parsing

PhD. Nguyen Thi Mai Trang



Content

Parsing
problem

Context-free
grammars

Some
grammar rules
for English

Treebanks

CKY parsing

Parsing problem

- What counts as a word?
- **Corpus** (plural **corpora**), a computer-readable collection of text or speech
- Ex: the Brown corpus is a million-word collection of samples from 500 written English texts from different genres (newspaper, fiction, non-fiction, academic, etc.)

How many words are in the following Brown sentence?

He stepped out into the hall, was delighted to encounter a water brother.

=> 13 words if we don't count punctuation marks as words, 15 if we count punctuation

Formal grammars

- A **grammar** is a powerful tool for describing and analyzing languages. It is a set of rules by which valid sentences in a language are constructed.

sentence	->	<subject> <verb-phrase> <object>
subject	->	This Computers I
verb-phrase	->	<adverb> <verb> <verb>
adverb	->	never
verb	->	is run am tell
object	->	the <noun> a <noun> <noun>
noun	->	university world cheese lies

- **Formal grammars** are a tool for syntax, not semantics.
- Types of Formal Grammars
 - Regular Grammars
 - Context-Free Grammars
 - Context-Sensitive Grammars
 - Unrestricted Grammars

Definitions

<i>grammar</i>	a set of rules by which valid sentences in a language are constructed.
<i>nonterminal</i>	a grammar symbol that can be replaced/expanded to a sequence of symbols.
<i>terminal</i>	an actual word in a language; these are the symbols in a grammar that cannot be replaced by anything else. "terminal" is supposed to conjure up the idea that it is a dead-end—no further expansion is possible.
<i>production</i>	a grammar rule that describes how to replace/exchange symbols. The general form of a production for a nonterminal is:

$$X \rightarrow Y_1 Y_2 Y_3 \dots Y_n$$

The nonterminal X is declared equivalent to the concatenation of the symbols $Y_1 Y_2 Y_3 \dots Y_n$. The production means that anywhere where we encounter X , we may replace it by the string $Y_1 Y_2 Y_3 \dots Y_n$. Eventually we will have a string containing nothing that can be expanded further, i.e., it will consist of only terminals. Such a string is called a *sentence*. In the context of programming languages, a sentence is a syntactically correct and complete program.

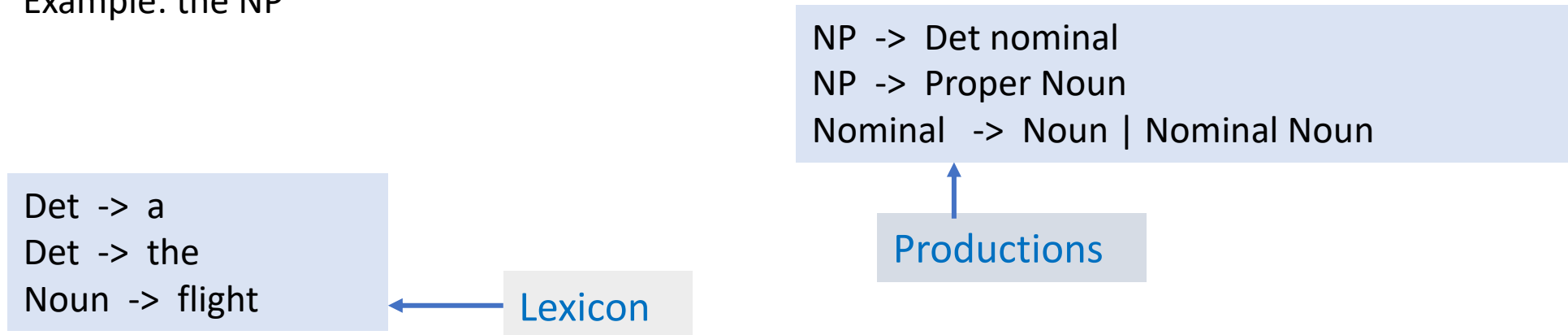
<i>derivation</i>	a sequence of applications of the rules of a grammar that produces a finished string of terminals. A <i>leftmost derivation</i> is where we always substitute for the leftmost nonterminal as we apply the rules (we can similarly define a rightmost derivation). A derivation is also called a <i>parse</i> .
<i>start symbol</i>	a grammar has a single nonterminal (the start symbol) from which all sentences derive:

$$S \rightarrow X_1 X_2 X_3 \dots X_n$$

	All sentences are derived from S by successive replacement using the productions of the grammar.
<i>null symbol</i> ϵ	it is sometimes useful to specify that a symbol can be replaced by nothing at all. To indicate this, we use the null symbol ϵ , e.g., $A \rightarrow B \mid \epsilon$.
<i>BNF</i>	a way of specifying programming languages using formal grammars and production rules with a particular form of notation (Backus-Naur form).

Context-Free Grammars

- A **context-free grammar (CFG)** consists of a set of **rules** or **productions**, each of which expresses the ways that symbols of the language can be grouped and ordered together, and a lexicon of words and symbols.
- CFG are also called **phrase-structure grammars**, and the formalism is equivalent to **Backus-Naur form**, or BNF.
- A CFG consists of:
 - rules or productions – expressing the way in which symbols of the language can be grouped together, and
 - a lexicon of words and symbols.
- Example: the NP



Symbols in CFG

The symbols that are used in a CFG are divided into two classes:

- The symbols that correspond to words in the language (“the”, “nightclub”) are called terminal symbols; the lexicon is the set of rules that introduce these terminal symbols.
- The non-terminal symbols that express abstractions over the terminals. In each context-free rule, the item to the right of the arrow ($\rightarrow$) is an ordered list of one or more terminals and non-terminals; to the left of the arrow is a single non-terminal symbol expressing some cluster or generalization.

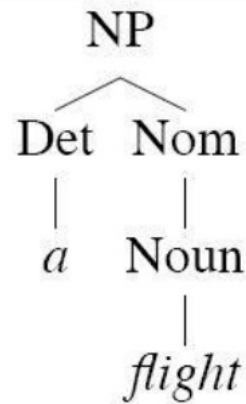
Det $\rightarrow$ a

Det $\rightarrow$ the

Noun $\rightarrow$ flight

Derivations in CFG

- A CFG can be thought of in two ways •
 - A device for **generating** sentences
 - A device for **assigning structure** to a sentence



Parse tree derived for the string “a flight”

L₀ Grammar

Summarize the grammar rules, which are called L₀.

Note that we can use the or-symbol | to indicate that a non-terminal has alternate possible expansions.

Noun → *flights* | *flight* | *breeze* | *trip* | *morning*
Verb → *is* | *prefer* | *like* | *need* | *want* | *fly* | *do*
Adjective → *cheapest* | *non-stop* | *first* | *latest*
 | *other* | *direct*
Pronoun → *me* | *I* | *you* | *it*
Proper-Noun → *Alaska* | *Baltimore* | *Los Angeles*
 | *Chicago* | *United* | *American*
Determiner → *the* | *a* | *an* | *this* | *these* | *that*
Preposition → *from* | *to* | *on* | *near* | *in*
Conjunction → *and* | *or* | *but*

The lexicon for L₀

L₀ Grammar

Grammar Rules	Examples
$S \rightarrow NP VP$	I + want a morning flight
$NP \rightarrow$ $Pronoun$ $$ $Proper-Noun$ $$ $Det Nominal$	- I - Los Angeles - a + flight
$Nominal \rightarrow$ $Nominal Noun$ $$ $Noun$	- morning + flight - flights
$VP \rightarrow$ $Verb$ $$ $Verb NP$ $$ $Verb NP PP$ $$ $Verb PP$	- do - want + a flight - leave + Boston + in the morning - leaving + on Thursday
$PP \rightarrow Preposition NP$	from + Los Angeles

The grammar for L₀, with example phrases for each rule.

Formal Definition Of Context-free Grammar

- A context-free grammar G is defined by four parameters: N, Σ, R, S (technically it is a “4-tuple”)

N a set of **non-terminal symbols** (or **variables**)

Σ a set of **terminal symbols** (disjoint from N)

R a set of **rules** or productions, each of the form $A \rightarrow \beta$,
where A is a non-terminal,

β is a string of symbols from the infinite set of strings $(\Sigma \cup N)^*$

S a designated **start symbol** and a member of N

Chomsky Normal Form

- A formal language is defined as a (possibly infinite) set of strings of words.
- A CFG is said to be in Chomsky Normal Form (CNF) if every production is one of these two forms:
 - 1. $A \rightarrow BC$ (body is two variables)
 - 2. $A \rightarrow a$ (body is a single terminal)
 - Where A, B, C are non-terminal and a is terminal
- Theorem: if L is a CFL, then $L - \{\epsilon\}$ has a CFG in CNF



Rules To Compute CNF

1. Eliminate Λ - Productions
2. Eliminate Unit Productions
3. Restricting the right side of productions to single terminal or string of two or more non-terminals (replace all mixed string with solid NTs)
4. Final step of CNF. (shorten the string of NT to length 2)

Example

- Convert following CFG to CNF:
- $S \rightarrow aX/Yb$
- $X \rightarrow S/\wedge$
- $Y \rightarrow bY/b$
- Step 1: Eliminate $\wedge$ -production:
 - Nullable production is $X \rightarrow \wedge$, new CFG without $\wedge$ -production is:
 $S \rightarrow aX/a/Yb$
 $X \rightarrow S$
 $Y \rightarrow bY/b$

Example

- Step 2: Eliminate Unit Production:

$S \rightarrow aX/a/Yb$

$X \rightarrow aX/a/Yb$

$Y \rightarrow bY/b$

- Step 3: Replace all mixed string with solid NT:

$S \rightarrow AX/YB/a$

$X \rightarrow AX/YB/a$

$Y \rightarrow BY/b$

$A \rightarrow a$

$B \rightarrow b$

Example

- Step 4: Shorten the string of NT to length 2
All NT strings on the RHS in the above CFG are already the required length.
So, CFG is in CNF.
- Result:
 - $S \rightarrow AX/YB/a$
 - $X \rightarrow AX/YB/a$
 - $Y \rightarrow BY/b$
 - $A \rightarrow a$
 - $B \rightarrow b$

Sentence-Level Constructions (1/3)

- Sentences have numerous constructions, but four are particularly common:
 1. Declarative structure
 2. Imperative structure
 3. Yes-no question structure
 4. Wh-question structure
- Sentences with **declarative structure** (e.g. “I prefer morning flights”) have a subject NP followed by a VP.

Examples:

The flight should be eleven a.m. tomorrow.

The return flight should leave around seven p.m.

I'd like to fly the coach discount class.

I want a flight from Ontario to Chicago.

We can model this structure with a rule: $S \rightarrow NP VP$

Sentence-Level Constructions (2/3)

- Sentences with **imperative structure** (e.g. “Show the lowest fare.”) often begin with a VP, and have no subject.

Examples

Show me the cheapest fare that serves lunch.

List all the flights between five and seven p.m.

Give me Sunday’s flights arriving in Las Vegas from New York City.

- We can model this structure with a rule: $S \rightarrow VP$
- Sentences with **yes-no question structure** (e.g. “Do any of these flights have stops ?”) is used to ask questions. They typically begin with an auxiliary verb followed by a subject NP , followed by a VP.

Examples:

- Does American flight eighteen twenty five serve dinner?
- Can you give me the same information on United?

- We can model this structure with a rule: $S \rightarrow Aux\ NP\ VP$

Sentence-Level Constructions (3/3)

- Sentences with **wh-structure** (e.g. “What airlines fly from Burbank to Denver ?”) have one constituent a wh-phrase, i.e. a phrase having a wh-word (who, when, whose, where, what, which, how, why).
- We have a a wh-subject question structure and a wh-non-subject question structure. The wh-subject structure is identical to the declarative structure, except that the first NP contains some wh-word.

Examples:

Which flights depart Burbank after noon and arrive in Denver by six p.m.?

Whose flights serve breakfast?

We can model this structure with a rule: $S \rightarrow \text{Wh-NP VP}$

- In sentences with **wh-non-subject question structure** the wh-phrase is not the subject of the sentence, thus the sentence includes another subject. In such sentences, the auxiliary appears before the subject NP, like in the yes-no question structures.

Examples:

What flights do you have from Burbank to Tacoma Washington?

We can model this structure with a rule: $S \rightarrow \text{Wh-NP Aux NP VP}$

Some Noun Phrase Rules

The following productions express that an NP (or noun phrase) can be composed of either a *ProperNoun* or a determiner (*Det*) followed by a *Nominal*; a *Nominal* in turn can consist of one or more Nouns.

$$\begin{aligned} NP &\rightarrow Det\ Nominal \\ NP &\rightarrow ProperNoun \\ Nominal &\rightarrow Noun \mid Nominal\ Noun \end{aligned}$$

Together, these describe **two kinds** of NPs.

- One that consists of a determiner followed by a nominal
- and another that says that proper names are NPs
- The third rule illustrates two things
 - An explicit disjunction
 - two kinds of nominals
 - A recursive definition
 - Same non-terminal on the right and left-side of the rule

Some Noun Phrase Rules

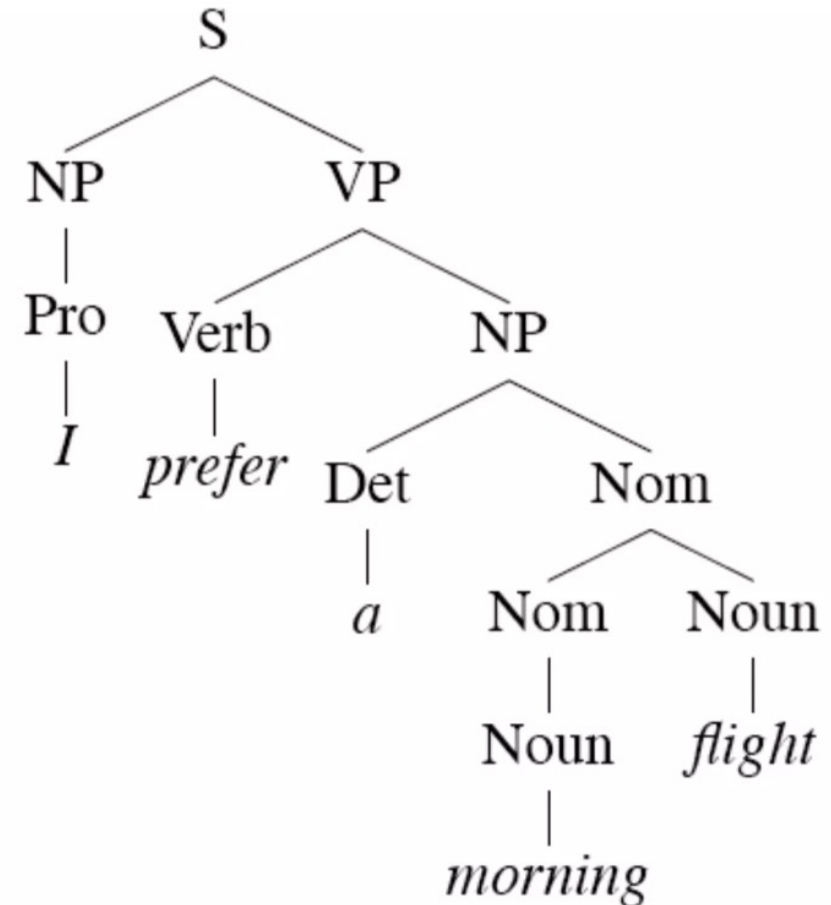
- The symbols that are used in a CFG are divided into two classes
 - **terminal** symbols: the symbols that correspond to words in the language
 - **non-terminals**: the symbols that express abstractions over these terminals
- A CFG can be thought of in two ways:
 - as a device for generating sentences
 - as a device for assigning a structure to a given sentence

Verb Phrase And Subcategorization

- VPs consist of a verb and a number of other constituents. These constituents include NPs and PPs:
 - $VP \rightarrow \text{Verb}$ Example: disappear
 - $VP \rightarrow \text{Verb NP}$ Example: prefer a morning flight
 - $VP \rightarrow \text{Verb NP PP}$ Example: leave Boston in the morning
 - $VP \rightarrow \text{Verb PP}$ Example: leaving on Thursday
- An entire embedded sentence may follow a verb. They are called sentential complements.
- To deal with sentential complements we add the rule: $VP \rightarrow \text{Verb S}$
- Another potential constituent of a VP is another VP. This happens for some verbs, e.g. want, try, would like, intend, need.
 - **Examples:** I want [_{VP} to fly from Dallas to Orlando]
Hello, I'm trying [_{VP} to find a flight from Dallas to Orlando]

Derivations

- A derivation is a sequence of rules applied to a string that accounts for that string
 - Covers all the elements in the string
 - Covers only the elements in the string



Coordination

- Phrases can be conjoined with conjunctions (e.g. and, or, but)
- Forms of coordinations:
 - For noun phrases **NP** → **NP and NP**
Please repeat $[_{NP}[_{NP} \text{the flights}] \text{ and } [_{NP} \text{the costs}]$
 - For nominals **Nominal** → **Nominal and Nominal**
Please repeat the $[_{Nom}[_{Nom} \text{flights}] \text{ and } [_{Nom} \text{costs}]]$
 - For verb phrases **VP** → **VP and VP**
What flights do you have $[_{VP}[_{VP} \text{leaving Denver}] \text{ and } [_{Nom} \text{arriving in San Francisco}]]$
 - For S conjunction
 $[_S[_S \text{I'm interested in a flight from Denver to Dallas}] \text{ and } [_S \text{I'm also interested in going to San Francisco}]]$
 - Coordination makes use of metarules: **X** → **X and X**

Treebanks

- Sufficiently robust grammars consisting of context-free grammar rules can be used to assign a parse tree to any sentence. This means that it is possible to build a corpus where every sentence in the collection is paired with a corresponding parse tree. Such a syntactically annotated corpus is called a **treebank**.
- Corpus in which each sentence is syntactically annotated with a parse tree
- The Penn Treebank uses corpora in English from Brown, Switchboard, ATIS and Wall Street Journal. There are treebanks for Chinese and Arabic as well.
- Other Treebanks: Prague Dependency Treebank for Czech, Negra Treebank for German, the Susanne Treebank for English.
- Universal Dependencies project (*Nivre et al., 2016*).

Treebanks

```
((S
  (NP-SBJ (DT That)
    (JJ cold) (, ,)
    (JJ empty) (NN sky) )
  (VP (VBD was)
    (ADJP-PRD (JJ full)
      (PP (IN of)
        (NP (NN fire)
          (CC and)
          (NN light) ))))
  (. .) ))
```

(a)

```
((S
  (NP-SBJ The/DT flight/NN )
  (VP should/MD
    (VP arrive/VB
      (PP-TMP at/IN
        (NP eleven/CD a.m/RB ))
      (NP-TMP tomorrow/NN )))))
```

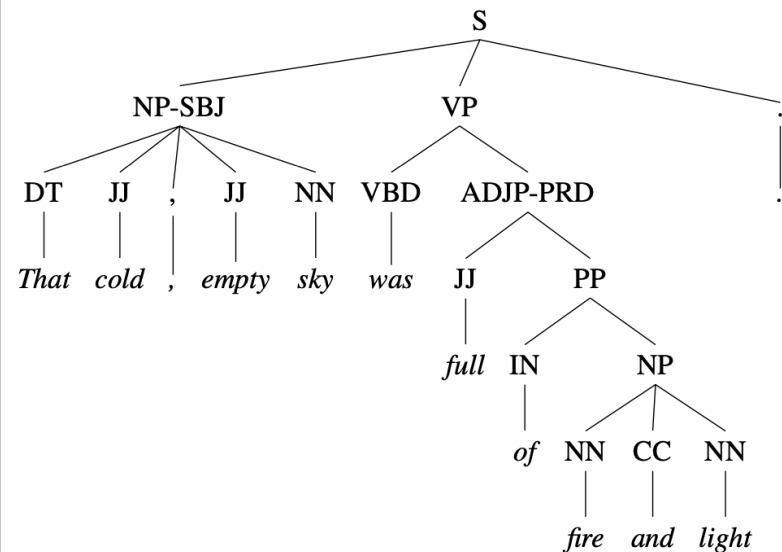
(b)

Parses from the Treebank project for (a) Brown and (b) ATIS sentences

Treebanks

```
((S
  (NP-SBJ (DT That)
    (JJ cold) (, ,)
    (JJ empty) (NN sky) )
  (VP (VBD was)
    (ADJP-PRD (JJ full)
      (PP (IN of)
        (NP (NN fire)
          (CC and)
          (NN light) ))))
  (. .) ))
(a)
```

```
((S
  (NP-SBJ The/DT flight/NN )
  (VP should/MD
    (VP arrive/VB
      (PP-TMP at/IN
        (NP eleven/CD a.m/RB ))
      (NP-TMP tomorrow/NN )))))
```



The tree corresponding to the
Brown corpus sentence

Using a Treebank as a Grammar

- The sentences in a treebank implicitly constitute a grammar of the language.

Grammar	Lexicon
$S \rightarrow NP VP .$	$DT \rightarrow the \mid that$
$S \rightarrow NP VP$	$JJ \rightarrow cold \mid empty \mid full$
$NP \rightarrow DT NN$	$NN \rightarrow sky \mid fire \mid light \mid flight \mid tomorrow$
$NP \rightarrow NN CC NN$	$CC \rightarrow and$
$NP \rightarrow DT JJ, JJ NN$	$IN \rightarrow of \mid at$
$NP \rightarrow NN$	$CD \rightarrow eleven$
$VP \rightarrow MD VP$	$RB \rightarrow a.m.$
$VP \rightarrow VBD ADJP$	$VB \rightarrow arrive$
$VP \rightarrow MD VP$	$VBD \rightarrow was \mid said$
$VP \rightarrow VB PP NP$	$MD \rightarrow should \mid would$
$ADJP \rightarrow JJ PP$	
$PP \rightarrow IN NP$	
$PP \rightarrow IN NP RB$	

CFG grammar rules and lexicon from the treebank sentences

Using a Treebank as a Grammar

The grammar used to parse the Penn Treebank is relatively flat, resulting in very many and very long rules.

For example among the approximately 4,500 different rules for expanding VP are separate rules for PP sequences of any length, and every possible arrangement of verb arguments:

VP → VBD PP

VP → VBD PP PP

VP → VBD PP PP PP

VP → VBD PP PP PP PP

VP → VB ADVP PP

VP → VB PP ADVP

VP → ADVP VB PP

as well as even longer rules, such as these two:

VP → VBP PP PP PP PP PP ADVP PP

VP → VBN PP ADVP PP PP SBAR

Rules and Sentences

VP → VBP PP PP PP PP PP ADVP PP

VP → VBN PP ADVP PP PP SBAR

Two of those rules, come from the following two VPs:

- This mostly happens because we [_{VP} go from football in the fall to lifting in the winter to football again in the spring.]
- Put more simply, GOP candidates for president are [_{VP} looked on more kindly by voters than Republican candidates for the Senate when the prisoner's dilemma is more severe].

Grammar Rules For NP Extracted From Treebanks

Some of the many thousands of NP rules include:

NP → DT JJ NN

NP → DT JJ NN NN

NP → DT JJ JJ NN

NP → DT JJ CD NNS

NP → RB DT JJ NN NN

NP → RB DT JJ JJ NNS

NP → DT JJ JJ NNP NNS

NP → DT NNP NNP NNP NNP JJ NN

NP → DT JJ NNP CC JJ JJ NN NNS

NP → RB DT JJS NN NN SBAR

NP → DT VBG JJ NNP NNP CC NNP

NP → DT JJ NNS , NNS CC NN NNS NN

NP → DT JJ JJ VBG NN NNP NNP FW NNP

NP → NP JJ , JJ " SBAR " NNS

CKY Parsing: A Dynamic Programming Approach

Dynamic programming provides a powerful framework for addressing the problems caused by ambiguity in grammars. a dynamic programming approach systematically fills in a table of solutions to subproblems. The complete table has the solution to all the subproblems needed to solve the problem as a whole

Cocke-Kasami-Younger parsing algorithm:

- (Relatively) efficient bottom-up parsing algorithm based on tabulating substring parses to avoid repeated work.
- Approach:
 - Use a CNF grammar
 - Build an $(n+1) \times (n+1)$ matrix to store subtrees
 - Upper triangular portion
 - Incrementally build parse spanning whole input string

Conversion To Chomsky Normal Form

- The CKY algorithm requires grammars to first be in **Chomsky Normal Form (CNF)**.
- The entire conversion process can be presented as follows:
 - Copy all conforming rules to the new grammar unchanged.
 - Convert terminals within rules to dummy non-terminals.
 - Convert unit productions.
 - Make all rules binary and add them to new grammar.

Conversion To Chomsky Normal Form

$\mathcal{L}_1$ Grammar	$\mathcal{L}_1$ in CNF
$S \rightarrow NP VP$	$S \rightarrow NP VP$
$S \rightarrow Aux NP VP$	$S \rightarrow X1 VP$
	$X1 \rightarrow Aux NP$
$S \rightarrow VP$	$S \rightarrow book \mid include \mid prefer$
	$S \rightarrow Verb NP$
	$S \rightarrow X2 PP$
	$S \rightarrow Verb PP$
	$S \rightarrow VP PP$
$NP \rightarrow Pronoun$	$NP \rightarrow I \mid she \mid me$
$NP \rightarrow Proper-Noun$	$NP \rightarrow TWA \mid Houston$
$NP \rightarrow Det Nominal$	$NP \rightarrow Det Nominal$
$Nominal \rightarrow Noun$	$Nominal \rightarrow book \mid flight \mid meal \mid money$
$Nominal \rightarrow Nominal Noun$	$Nominal \rightarrow Nominal Noun$
$Nominal \rightarrow Nominal PP$	$Nominal \rightarrow Nominal PP$
$VP \rightarrow Verb$	$VP \rightarrow book \mid include \mid prefer$
$VP \rightarrow Verb NP$	$VP \rightarrow Verb NP$
$VP \rightarrow Verb NP PP$	$VP \rightarrow X2 PP$
	$X2 \rightarrow Verb NP$
$VP \rightarrow Verb PP$	$VP \rightarrow Verb PP$
$VP \rightarrow VP PP$	$VP \rightarrow VP PP$
$PP \rightarrow Preposition NP$	$PP \rightarrow Preposition NP$

$\mathcal{L}_1$ Grammar and its conversion to CNF.

CKY Recognition

Figure shows how the five cells of column 5 of the table are filled after the word *Houston* is read.

- The arrows point out the two spans that are being used to add an entry to the table.
- The action in cell [0, 5] indicates the presence of three alternative parses for this input, one where the *PP* modifies the *flight*, one where it modifies the booking, and one that captures the second argument in the original *VP* → *Verb NP PP* rule

<i>Book</i>	<i>the</i>	<i>flight</i>	<i>through</i>	<i>Houston</i>
S, VP, Verb Nominal, Noun [0,1]	[0,2]	S,VP,X2 [0,3]	[0,4]	S,VP,X2 [0,5]
	Det [1,2]	NP [1,3]	[1,4]	NP [1,5]
		Nominal, Noun [2,3]	[2,4]	Nominal [2,5]
			Prep [3,4]	PP [3,5]
				NP, Proper- Noun [4,5]

Diagram illustrating the CKY recognition process for the sentence "Book the flight through Houston". The diagram shows a 5x5 parse table and a corresponding 5x5 grid of blue boxes. Arrows indicate the spans used to fill the cells in column 5. A long arrow at the bottom points to the right, indicating the sequence of words processed.

Completed parse table for *Book the flight through Houston*

CKY Recognition

```
function CKY-PARSE(words, grammar) returns table
```

```
  for  $j \leftarrow$  from 1 to LENGTH(words) do
```

```
    for all  $\{A \mid A \rightarrow \text{words}[j] \in \text{grammar}\}$ 
```

```
       $\text{table}[j-1, j] \leftarrow \text{table}[j-1, j] \cup A$ 
```

```
    for  $i \leftarrow$  from  $j-2$  down to 0 do
```

```
      for  $k \leftarrow i+1$  to  $j-1$  do
```

```
        for all  $\{A \mid A \rightarrow BC \in \text{grammar} \text{ and } B \in \text{table}[i, k] \text{ and } C \in \text{table}[k, j]\}$ 
```

```
           $\text{table}[i, j] \leftarrow \text{table}[i, j] \cup A$ 
```

The CKY algorithm.

CKY Parsing

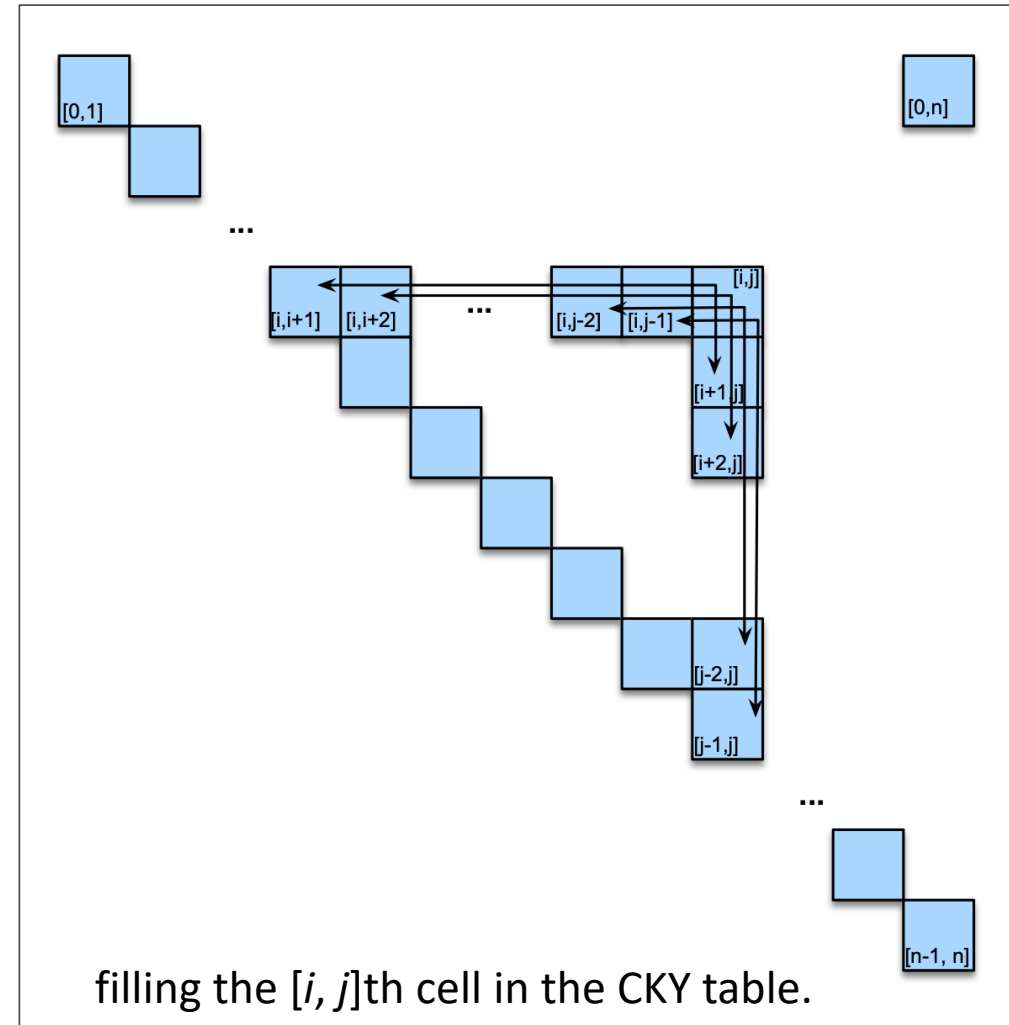
- The CKY algorithm is a recognizer, not a parser.
- Table fills:
 - Column-by-column
 - Left-to-right
 - Bottom-to-top
- Why?
 - Necessary info available (below and left)
 - Allows online sentence analysis
 - Works across input string as it arrives

CKY Parsing

Enhancing CKY for Parsing

Two essential modifications to transform CKY into a parser capable of generating all possible parses:

- Augmenting table entries: Pairing non-terminals with pointers to their derivation entries.
- Permitting multiple versions: Allowing varied representations of the same non-terminal in the table.



From Recognition to Parsing

Limitations of current recognition algorithm:

- Only stores non-terminals in cell
 - Not rules or cells corresponding to RHS
- Stores SETS of non-terminals
 - Can't store multiple rules with same LHS

Parsing solution:

- All repeated versions of non-terminals
- Pair each non-terminal with pointers to cells
 - Backpointers
- Last step: construct trees from back-pointers in $[0, n]$

Natural Language Processing

Chapter 6: Lexical Semantics

PhD. Nguyen Thi Mai Trang



Content

- Word senses
- Relations between senses
 - Synonymy and antonymy
 - Hyponymy
 - WordNet
- Word sense disambiguation (WSD)
- Methods for WSD
 - Supervised WSD
 - Dictionary-based methods
 - Bootstrapping

Word Senses

- A sense (or word sense) is a discrete representation of one aspect of the meaning of a word
- “Word” are having different meanings based on the context of the usage in a sentence

We went to see a **play** at the theater

The children went out to **play** in the park

Word Senses

- Computational representation of word meanings using embeddings (Word2Vec, GloVe).
- Limitations in defining word senses using embeddings.
- The role of contextual embeddings like BERT.
 - BERT offers an embedding that represents word meanings within their textual context.
 - Traditional word embeddings (e.g., word2vec, GloVe) represent words as fixed points in semantic space.
 - Contextual embeddings, on the other hand, capture word meanings based on their surroundings in a sentence.
- Significance of contextual embeddings in word sense disambiguation (WSD).

WordNet

- **WordNet** is a lexical database for the English language based on conceptual look-up
- George A. Miller who began the WordNet project in the mid 1980s.
- Organizes lexical information in terms of word meaning rather than word form.
- WordNet can also be used as a **thesaurus**

Synonymy

- Word that have the same meaning in some or all contexts
 - Couch/sofa
 - Big/large
 - Smart/intelligent
 - Happy/joyful
 - Laugh/chuckling
- Two leexems are synonyms if they can be succesfully substituted for each other in all situations
 - If so they have the same propositional meaning

Synonymy Relates Senses Not Words

- Consider big and large
- Are they synonyms?
 - How **big** is that plane?
 - Would i be flying on a **large** or a small plane?
- How about:
 - Miss Nelson, for instance, became a kind of **big** sister to Benjamin.
 - Miss Nelson, for instance, became a kind of **large** sister to Benjamin.
- Why?
 - ✓ **Big** has a sense that means being older, or grown up
 - ✓ **Large** lacks this sense

Antonyms

- Senses that are opposites with respect to one feature of their meaning
- Otherwise, they are very similar
 - Dark/light
 - Short/long
 - Big/small
 - Hot/cold
 - Up/down
- More formally: antonyms can
 - Define a binary opposition or an attribute at opposite ends of a scale (long/short, fast/slow)
 - Be reversives: rise/fall, up/down

Hyponyms

- A sense is a hyponym of another if the first sense is more specific, denoting a subclass of the other
 - Car is a hyponym of vehicle
 - Cat is a hyponym of animal
 - Cabbage is a hyponym of vegetable
- Conversely
 - Vehicle is a hypernym/superordinate of car
 - Animal is a hypernym of cat
 - Vegetable is a a hypernym of cabbage

Superordinate	Vehicle	Vegetable	animal	Furniture
Hyponym	Care	Cabbage	cat	table

Hypernymy

- Extensional
 - The class denoted by the superordinate
 - Extensionally includes class denoted by the hyponym
- Entailment
 - **A sense A is a hyponym of sense B if being an A entails being a B**
- Hyponymy is usually transitive
 - A hypo B and B hypo C entails A hypo C

Wordnet: A Database Of Lexical Relations

- English WordNet consists of three separate databases, one each for nouns and verbs and a third for adjectives and adverbs; closed class words are not included.

Category	Unique forms
Noun	117097
Verb	11488
Adjective	22141
Adverb	4601

- WordNet: <http://wordnetweb.princeton.edu/perl/webwn>

WordNet Entries

The noun “bass” has 8 senses in WordNet.

1. bass¹ - (the lowest part of the musical range)
2. bass², bass part¹ - (the lowest part in polyphonic music)
3. bass³, basso¹ - (an adult male singer with the lowest voice)
4. sea bass¹, bass⁴ - (the lean flesh of a saltwater fish of the family Serranidae)
5. freshwater bass¹, bass⁵ - (any of various North American freshwater fish with lean flesh (especially of the genus *Micropterus*))
6. bass⁶, bass voice¹, basso² - (the lowest adult male singing voice)
7. bass⁷ - (the member with the lowest range of a family of musical instruments)
8. bass⁸ - (nontechnical name for any of numerous edible marine and freshwater spiny-finned fishes)

A portion of the WordNet 3.0 entry for the noun *bass*.

Sense Relations In Wordnet

- Some of the noun relations in WordNet

Relation	Also Called	Definition	Example
Hypernym	Superordinate	From concepts to superordinates	<i>breakfast</i> ¹ $\rightarrow$ <i>meal</i> ¹
Hyponym	Subordinate	From concepts to subtypes	<i>meal</i> ¹ $\rightarrow$ <i>lunch</i> ¹
Instance Hypernym	Instance	From instances to their concepts	<i>Austen</i> ¹ $\rightarrow$ <i>author</i> ¹
Instance Hyponym	Has-Instance	From concepts to their instances	<i>composer</i> ¹ $\rightarrow$ <i>Bach</i> ¹
Part Meronym	Has-Part	From wholes to parts	<i>table</i> ² $\rightarrow$ <i>leg</i> ³
Part Holonym	Part-Of	From parts to wholes	<i>course</i> ⁷ $\rightarrow$ <i>meal</i> ¹
Antonym		Semantic opposition between lemmas	<i>leader</i> ¹ $\iff$ <i>follower</i> ¹
Derivation		Lemmas w/same morphological root	<i>destruction</i> ¹ $\iff$ <i>destroy</i> ¹

Sense Relations in WordNet

- Some verb relations in WordNet

Relation	Definition	Example
Hypernym	From events to superordinate events	<i>fly</i> ⁹ → <i>travel</i> ⁵
Troponym	From events to subordinate event	<i>walk</i> ¹ → <i>stroll</i> ¹
Entails	From verbs (events) to the verbs (events) they entail	<i>snore</i> ¹ → <i>sleep</i> ¹
Antonym	Semantic opposition between lemmas	<i>increase</i> ¹ ⇔ <i>decrease</i> ¹

Word Sense Disambiguation (WSD)

- Given
 - A word in context
 - A fixed inventory of potential word senses
- Decide which sense of the word this is
 - English-to-Vietnamese machine translation
 - Inventory is set of Vietnamese translations
 - Speech synthesis
 - Inventory is homographs with different pronunciations like bass and bow
 - Automatic indexing of medical articles
 - MeSH (Medical Subject Heading) thesaurus entries

Two Variants Of WSD

- Lexical sample task
 - Small pre-selected set of target words
 - And inventory of senses for each word
- All-words task
 - Every word in an entire text
 - A lexicon with senses for each word
 - Like part-of-speech tagging
 - Except each lemma has it own tagset

Methods

There are four conventional approaches to WSD:

- Dictionary- and knowledge-based methods
- Supervised methods
- Semi-supervised (or minimally-supervised) methods
- Unsupervised methods

Supervised WSD

- Based on labeled training set
- The learning system has
 - A training set of feature-encoded input AND
 - Their appropriate sense label (category)

There are following methods in Supervised WSD

1. Decision List
2. Decision Tree
3. Naive Bayes
4. Instance-Based Learning
5. Majority Voting

Unsupervised WSD

- Based on unlabeled training set
- The learning system has
 - A training set of feature-encoded input AND
 - **NOT** their appropriate sense label (category)

Main approaches of unsupervised are as follow:

1. Context Clustering
2. Co-occurrence Graph
3. Spanning tree based approach
4. Word Clustering

Dictionary-based Method For WSD

- It is an approach leverages information from a dictionary, thesaurus, or lexical resource to determine the correct sense of a word within a particular context.
- The most well-known dictionary-based WSD algorithm is the Lesk algorithm, which works as follows:
 1. Given a word to be disambiguated, the algorithm looks up its definition in the dictionary.
 2. The algorithm then identifies all of the other words in the context (e.g., the preceding and following words in a sentence) and looks up their definitions in the dictionary as well.
 3. The algorithm then computes a similarity score between the definition of the word to be disambiguated and the definitions of all of the other words in the context.
 4. The algorithm then selects the sense of the word to be disambiguated that has the highest similarity score to the other words in the context.
- Pros: simple and easy to implement
- Cons:
 - they are limited by the quality of the dictionary
 - do not take into account the context of the word to be disambiguated beyond the immediate surrounding words.

Semi-supervised Bootstrapping

- What if you don't have enough data to train a system.
- Bootstrap
 - Pick a word that you as an analyst think will co-occur with your target word in particular sense
 - Grep through your corpus for your target word and the hypothesized word
 - Assume that the target tag is the right one

Bootstrapping

- Given a set labelled examples L , a set of unlabeled examples U and a classifier c :
 - Choose N examples from U and add them to U'
 - Train c on L and label U'
 - Select K most confidently labelled instances from U' and assign them to L
 - Repeat until U or K is empty.

Evaluation

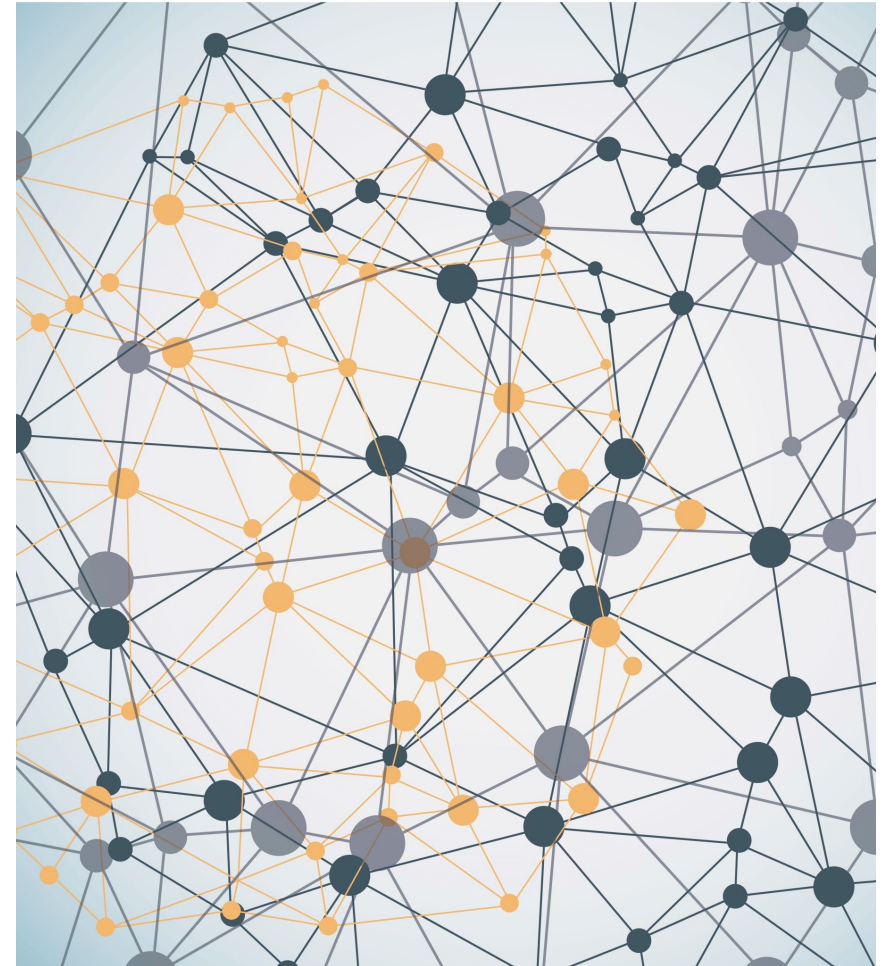
The evaluation of WSD systems requires a test corpus hand-annotated with the target or correct senses and assumes that such a corpus can be constructed. Two main performance measures are used:

- **Precision:** the fraction of system assignments made that are correct
- **Recall:** the fraction of total word instances correctly assigned by a system

Natural Language Processing

Chapter 7: Information Extraction

PhD. Nguyen Thi Mai Trang

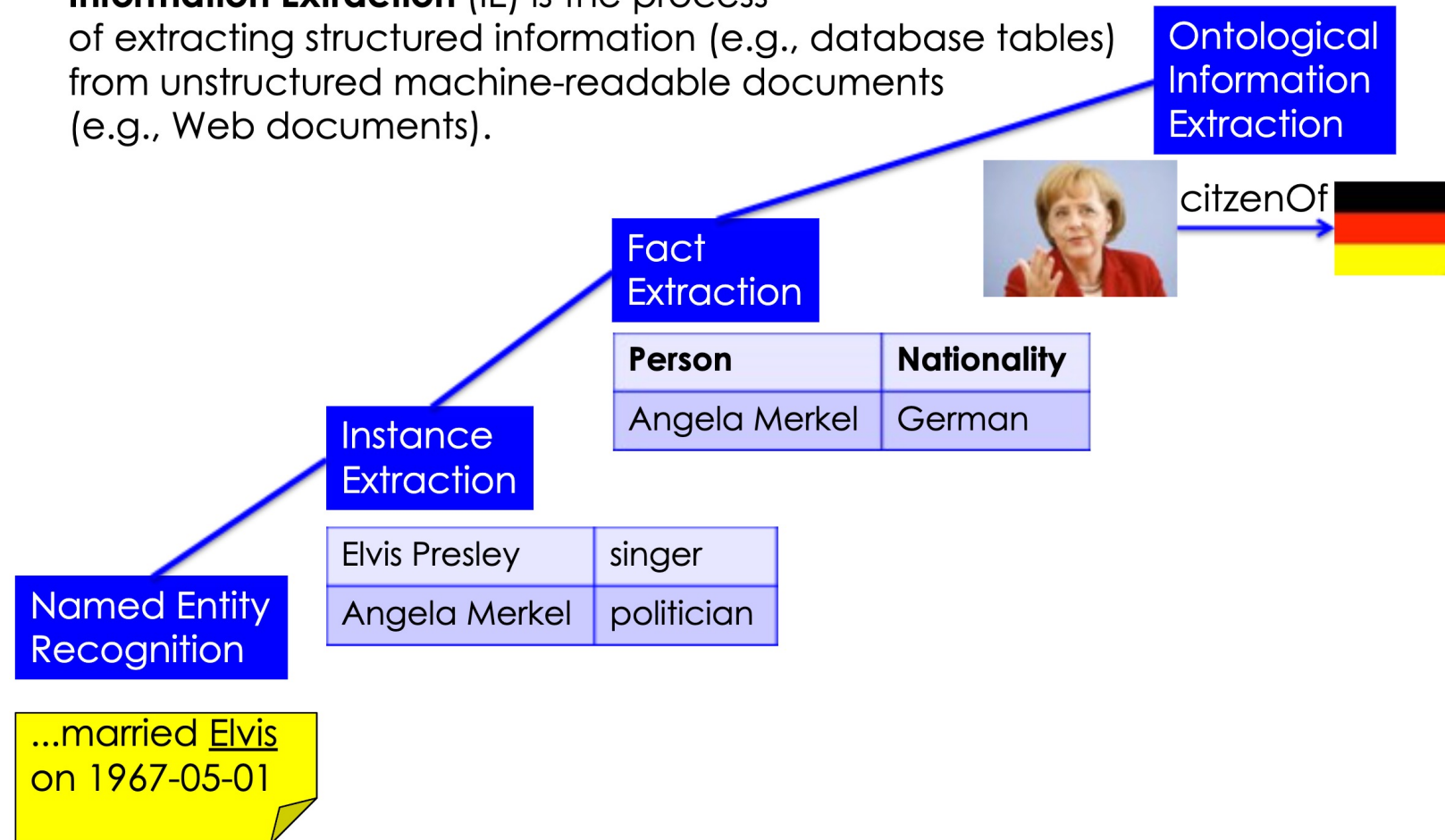


Content

- Introduction to information extraction
- Named entity recognition (NER)
- Relation extraction
- Other types of extraction
- Projects

Introduction

Information Extraction (IE) is the process of extracting structured information (e.g., database tables) from unstructured machine-readable documents (e.g., Web documents).



Introduction

Information extraction (IE) systems

- Find and understand limited relevant parts of texts
- Gather information from many pieces of text
- Produce a structured representation of relevant information:
 - a knowledge base

Goals:

- Organize information so that it is useful to people
- Put information in a semantically precise form that allows further inferences to be made by computer algorithms

Named Entity Recognition

- **Named Entity Recognition (NER)** is the process of finding entities (name, locations, dates,...) in a text.

Ho Chi Minh was born in 1890 in Nghe An province, in a village of Kim Lien

We can extract different types of entities:

- Entities for which we have an exhaustive dictionary (closed set extraction)
- Proper names (open set extraction)
- Entities that follow a certain pattern

Why NER?

- Coreference Resolution / Entity Linking
- Relation Extraction
- Knowledge Base Construction
- Web Query Understanding
- Question Answering
- ...

NER: Regular Expressions

A regular expression (regex) over a set of symbol Σ is:

1. the empty string
2. The string consisting of an element of Σ (a single character)
3. The string AB where A and B are regular expressions (**concatenation**)
4. A string of the form $(A|B)$, where A and B are regular expressions (**alternation**)
5. String of the form $(A)^*$, where A is a regular expression (**Kleene star**)

NER: Regular Expressions

A regular expression (regex) over a set of symbol Σ is:

1. the empty string
2. The string consisting of an element of Σ (a single character)
3. The string AB where A and B are regular expressions (**concatenation**)
4. A string of the form $(A|B)$, where A and B are regular expressions (**alternation**)
5. String of the form $(A)^*$, where A is a regular expression (**Kleene star**)

NER: Regular Expressions

Given an ordered set of symbols Σ , we define

$[x-y]$ for two symbols x and y , $x < y$, to be the alternation

$x|\dots|y$ (meaning: any of the symbols in the range)

NER: Regular Expressions

$A \mid B$ Either A or B

A^* Zero or more occurrences of A

A^+ One or more occurrences of A

$A\{x,y\}$ x to y occurrences of A

$A?$ an optional A

$[a-z]$ One of the characters in the range

$.$ An arbitrary symbol

NER: Regular Expressions

When using regular expression in a program, it is common to name them:

```
String digits="[0-9]+";
```

```
String separator="[ -]";
```

```
String pattern=digits+separator+digits;
```

NER: Normalization

Problem 1: we might extract strings that differ only slightly and mean the same thing.

Elvis Presley	singer
ELVIS PRESLEY	singer

Problem 2: we might extract literals (numbers, dates, etc.) that mean the same.

Elvis Presley	1935-01-08
ELVIS PRESLEY	08/01/35

Named Entity Recognition Task

- NER is a task of Natural Language Processing that involves identifying and classifying name entities in a text into predefined categories (names, locations, organizations, etc.).
- Goal:
 - Extract structured information from unstructured text data
 - Represent information in a machine readable format.
- Different techniques are seen:
 - Closed-set extraction (if the set of entities is known)
 - Extraction with Regular Expressions (if the entities follow a pattern)
- Need Normalization in addition

Sequence labeling for NER

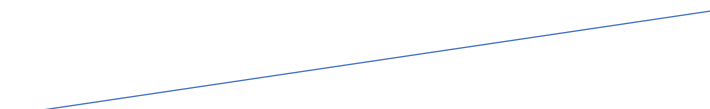
Traditional Named Entity Recognition Systems

- Heavy reliance on corpus-specific human labeling
- Training sequence models is slow

NER can be framed as a sequence labeling task in natural language processing. In this context, the goal is to label each word or token in a given text sequence with its corresponding entity type.

Encoding Labels For Sequence

BIOES labeling schema:

- The O
- European B-ORG 
- Commission E-ORG 
- said O
- on O
- Thursday O (where "O" represents words that are not part of any named entity.)
- it O
- disagreed O
- with O
- German S-MISC 

Begin of Entity

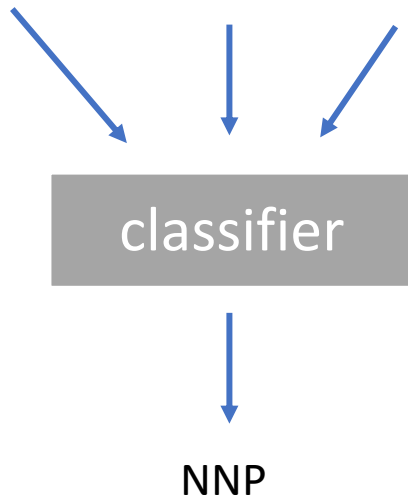
End of Entity

Singleton Entity

Sequence Labeling As Classification

Classify each token independently but use as input features, information about the surrounding tokens (sliding window).

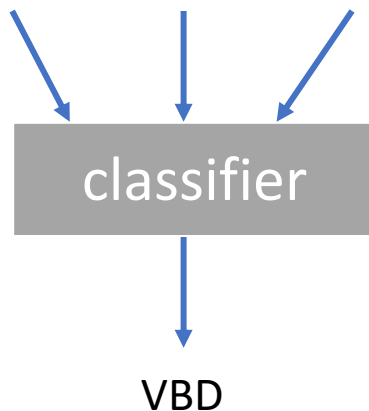
John saw the saw and decided to take it to the table.



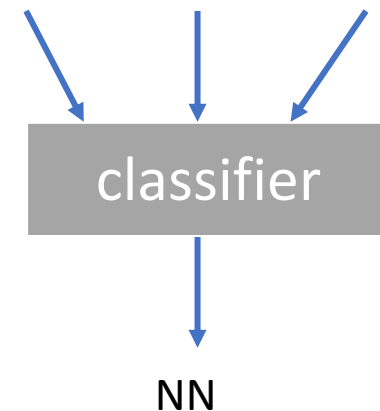
Sequence Labeling As Classification

Classify each token independently but use as input features, information about the surrounding tokens (sliding window).

John saw the saw and decided to take it to the table.



. . .



Sequence Labeling Models

To perform sequence labeling for NER, various machine learning and deep learning models can be used

- Conditional Random Fields (CRF)
- Recurrent Neural Networks (RNNs)
- Long Short-Term Memory (LSTM) networks
- Transformer-based models like BERT.

Sequence Models are statistical models of whole token sequences that effectively label sub-sequences

Training Data & Inference

Training Data: NER models are trained on labeled data where words in a text sequence are annotated with their corresponding entity labels.

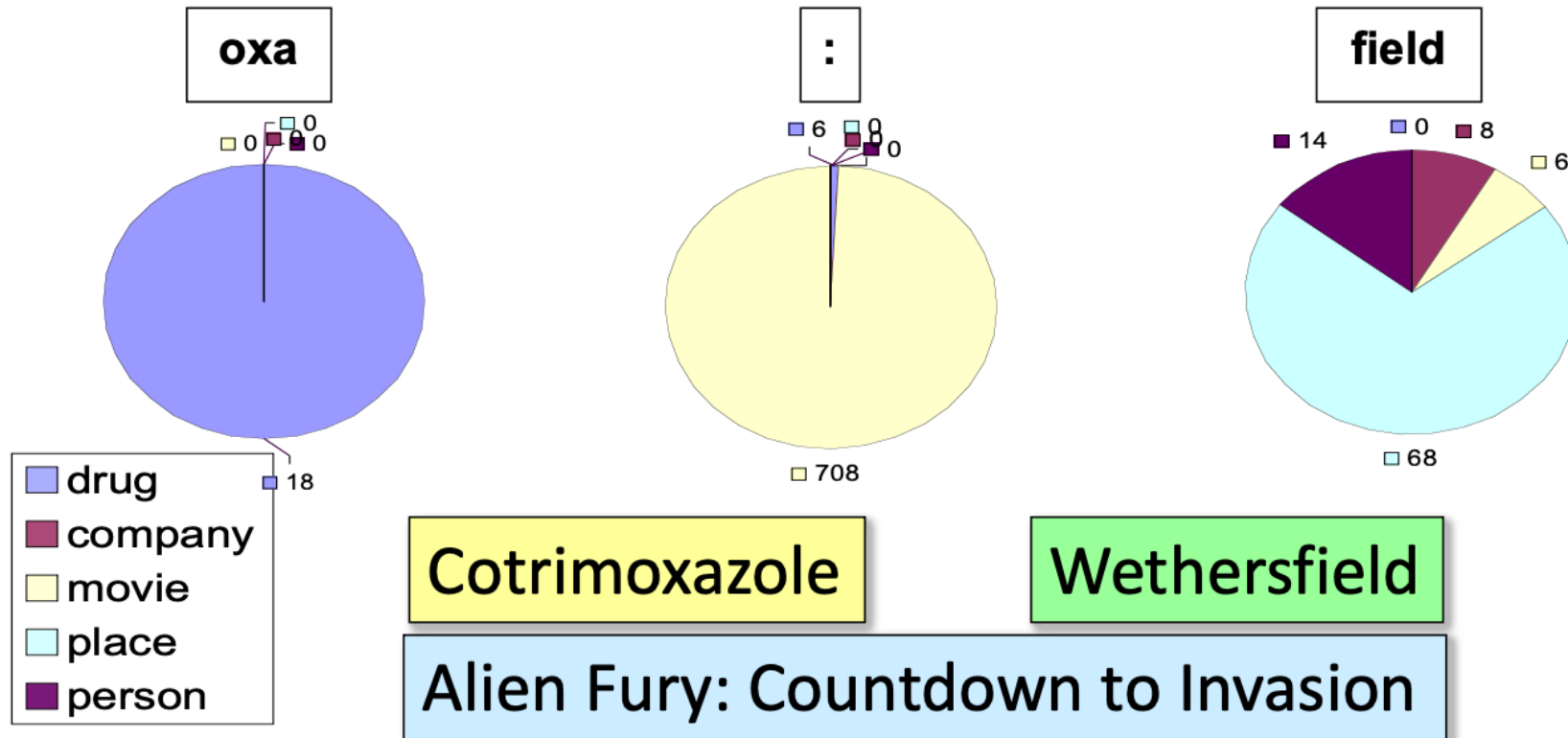
- Input sequence: ["Apple", "Inc.", "is", "headquartered", "in", "Cupertino"]
- Entity labels: ["ORG", "ORG", "O", "O", "O", "LOC"]

Inference: Once the NER model is trained, it can be used for inference on new, unlabeled text. It will predict the entity labels for each word in the text, effectively identifying and categorizing named entities.

Features for NER

- Words
 - Current word (essentially like a learned dictionary)
 - Previous/next word (context)
- Other kinds of inferred linguistic classification
 - Part-of-speech tags
- Label context
 - Previous (and perhaps next) label

Features: Word Substrings



Features For NER

- Word Shapes
 - Map words to simplified representation that encodes attributes such as length, capitalization, numerals, Greek letters, internal punctuation, etc.

Varicella-zoster	Xx-xxx
mRNA	xXXX
CPA1	XXXd

Evaluation of NER

- “correct” -> entity mention’s **boundary** is correct & predicted type **label** is correct

	correct	not correct
selected	tp	fp
not selected	fn	tn

Precision, Recall, F1 for NER

- When there are “boundary errors”:
 - First Bank of Chicago announced earnings ...
- This counts as both a FP (the system selects a wrong one) and a FN (the system misses a true one)
- Selecting nothing would have been better
- Some other metrics (e.g., MUC scorer) give partial credit (according to complex rules)

Precision, Recall

- Precision: % of selected items that are correct
- Recall: % of correct items that are selected

F-score: A Combined Measure

- A combined measure that assesses the P/R tradeoff is F measure (weighted harmonic mean)

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

- The harmonic mean is a very conservative average
- People usually use balanced F1 score
 - i.e., with $\beta = 1$ (that is, $\alpha = 1/2$): $F1 = 2PR/(P+R)$

Relation Extraction

- Relation extraction: finding and classifying semantic relations among entities mentioned in a text, like child-of (X is the child-of Y), or part-whole or geospatial relations.

Citing high fuel prices, [ORG United Airlines] said [TIME Friday] it has increased fares by [MONEY \$6] per round trip on flights to some cities also served by lower-cost carriers. [ORG American Airlines], a unit of [ORG AMR Corp.], immediately matched the move, spokesman [PER Tim Wagner] said. [ORG United], a unit of [ORG UAL Corp.], said the increase took effect [TIME Thursday] and applies to most routes where it competes against discount carriers, such as [LOC Chicago] to [LOC Dallas] and [LOC Denver] to [LOC San Francisco].

The text tells us, for example, that *Tim Wagner* is a spokesman for *American Airlines*, that *United* is a unit of *UAL Corp.*, and that *American* is a unit of *AMR*

Relation Extraction

Domain	$\mathcal{D} = \{a, b, c, d, e, f, g, h, i\}$
United, UAL, American Airlines, AMR	a, b, c, d
Tim Wagner	e
Chicago, Dallas, Denver, and San Francisco	f, g, h, i
Classes	
United, UAL, American, and AMR are organizations	$Org = \{a, b, c, d\}$
Tim Wagner is a person	$Pers = \{e\}$
Chicago, Dallas, Denver, and San Francisco are places	$Loc = \{f, g, h, i\}$
Relations	
United is a unit of UAL	$PartOf = \{\langle a, b \rangle, \langle c, d \rangle\}$
American is a unit of AMR	
Tim Wagner works for American Airlines	$OrgAff = \{\langle c, e \rangle\}$
United serves Chicago, Dallas, Denver, and San Francisco	$Serves = \{\langle a, f \rangle, \langle a, g \rangle, \langle a, h \rangle, \langle a, i \rangle\}$

A model-based view of the relations and entities in sample text

Relation Types: Freebase

Dataset from Wikipedia infoboxes, **Freebase** ([Bollacker et al., 2008](#)), now part of Wikidata ([Vrandećić and Krötzsch, 2014](#)), has relations between people and their nationality, or locations, and other locations they are contained in.

23 Million Entities, thousands of relations

Relation name	Size	Example
/people/person/nationality	281,107	John Dugard, South Africa
/location/location/contains	253,223	Belgium, Nijlen
/people/person/profession	208,888	Dusa McDuff, Mathematician
/people/person/place_of_birth	105,799	Edwin Hubble, Marshfield
/dining/restaurant/cuisine	86,213	MacAyo's Mexican Kitchen, Mexican
/business/business_chain/location	66,529	Apple Inc., Apple Inc., South Park, NC
/biology/organism_classification_rank	42,806	Scorpaeniformes, Order
/film/film/genre	40,658	Where the Sidewalk Ends, Film noir
/film/film/language	31,103	Enter the Phoenix, Cantonese
/biology/organism_higher_classification	30,052	Calopteryx, Calopterygidae
/film/film/country	27,217	Turtle Diary, United States
/film/writer/film	23,856	Irving Shulman, Rebel Without a Cause
/film/director/film	23,539	Michael Mann, Collateral
/film/producer/film	22,079	Diane Eskenazi, Aladdin
/people/deceased_person/place_of_death	18,814	John W. Kern, Asheville
/music/artist/origin	18,619	The Octopus Project, Austin
/people/person/religion	17,582	Joseph Chartrand, Catholicism
/book/author/works_written	17,278	Paul Auster, Travels in the Scriptorium
/soccer/football_position/players	17,244	Midfielder, Chen Tao
/people/deceased_person/cause_of_death	16,709	Richard Daintree, Tuberculosis
/book/book/genre	16,431	Pony Soldiers, Science fiction
/film/film/music	14,070	Stavisky, Stephen Sondheim
/business/company/industry	13,805	ATS Medical, Health care

Relation Types: WordNet

WordNet or other ontologies offer useful ontological relations that express hierarchical relations between words or concepts. WordNet is incomplete.

Ontological relations are missing for many words:

In WordNet 3.1	Not in WordNet 3.1
insulin progesterone	leptin pregnenolone
combustibility navigability	affordability reusability
HTML	XML
Google, Yahoo	Microsoft, IBM

Esp. for specific domains: restaurants, auto parts, finance

Relation Types: WordNet

WordNet or other ontologies offer useful ontological relations that express hierarchical relations between words or concepts. WordNet is incomplete.

Ontological relations are missing for many words:

In WordNet 3.1	Not in WordNet 3.1
insulin progesterone	leptin pregnenolone
combustibility navigability	affordability reusability
HTML	XML
Google, Yahoo	Microsoft, IBM

Esp. for specific domains: restaurants, auto parts, finance

Using Patterns to Extract Relations

- Lexico-syntactic patterns, first developed by [Hearst \(1992a\)](#), and therefore often called **Hearst patterns**.

Agar is a substance prepared from a mixture of red algae, such as Gelidium, for laboratory or industrial use.

suggests the following lexico-syntactic pattern

$$NP_0 \text{ such as } NP_1\{, NP_2 \dots, (and|or)NP_i\}, i \geq 1$$

implies the following semantics

$$\forall NP_i, i \geq 1, \text{hyponym}(NP_i, NP_0)$$

allowing us to infer

$$\text{hyponym}(\text{Gelidium}, \text{red algae})$$

Using Patterns to Extract Relations

- Five patterns [Hearst \(1992a, 1998\)](#) suggested for inferring the hyponym relation

NP {, NP}* {,} (and or) other NP _H	temples, treasures, and other important civic buildings
NP _H such as {NP,}* {(or and)} NP	red algae such as Gelidium
such NP _H as {NP,}* {(or and)} NP	such authors as Herrick, Goldsmith, and Shakespeare
NP _H {,} including {NP,}* {(or and)} NP	common-law countries , including Canada and England
NP _H {,} especially {NP,}* {(or and)} NP	European countries , especially France, England, and Spain

- Hand-built patterns have the advantage of high-precision and they can be tailored to specific domains.
- They are often low-recall, and it's a lot of work to create them for all possible patterns.

Relation Extraction Via Supervised Learning

- A fixed set of relations and entities is chosen
- A training corpus is hand-annotated with the relations and entities
- The annotated texts are then used to train classifiers to annotate an unseen test set.

Finding and classifying the relations among entities in a text

```
function FINDRELATIONS(words) returns relations  
  
  relations  $\leftarrow$  nil  
  entities  $\leftarrow$  FINDENTITIES(words)  
  forall entity pairs  $\langle e1, e2 \rangle$  in entities do  
    if RELATED?(e1, e2)  
      relations  $\leftarrow$  relations + CLASSIFYRELATION(e1, e2)
```

Semi-supervised Relation Extraction via Bootstrapping

Bootstrapping from seed entity pairs to learn relations

function BOOTSTRAP(*Relation R*) **returns** *new relation tuples*

tuples $\leftarrow$ Gather a set of seed tuples that have relation *R*

iterate

sentences $\leftarrow$ find sentences that contain entities in *tuples*

patterns $\leftarrow$ generalize the context between and around entities in *sentences*

newpairs $\leftarrow$ use *patterns* to identify more tuples

newpairs $\leftarrow$ *newpairs* with high confidence

tuples $\leftarrow$ *tuples* + *newpairs*

return *tuples*

Semi-supervised Relation Extraction via Bootstrapping

- Bootstrapping systems also assign confidence values to new tuples to avoid semantic drift.
- Confidence values for patterns are based on balancing two factors: **the pattern's performance** with respect to the current set of tuples and **the pattern's productivity** in terms of the number of matches it produces in the document collection.
- Given a document collection D , a current set of tuples T , and a proposed pattern p , we need to track two factors:
 - $hits(p)$: the set of tuples in T that p matches while looking in D
 - $finds(p)$: The total set of tuples that p finds in D
- Equation takes into account these factors

$$Conf_{RlogF}(p) = \frac{|hits(p)|}{|finds(p)|} \log(|finds(p)|)$$

- Equation for confidence in a new tuple.

$$Conf(t) = 1 - \prod_{p \in P'} (1 - Conf(p))$$

Evaluation Of Relation Extraction

- Supervised relation extraction systems are evaluated by using test sets with human- annotated, gold-standard relations and computing precision, recall, and F-measure
- Semi-supervised and unsupervised methods are much more difficult to evaluate. Because these methods use very large amounts of text, and as a result it's not possible to pre-annotate a gold set of correct instances of relations.
 - Compute approximate Precision.

$$\hat{p} = \frac{\text{\# of correctly extracted relation tuples in the sample}}{\text{total \# of extracted relation tuples in the sample.}}$$

Event Extraction

- The task of event extraction is to identify mentions of events in texts.
- Event extraction is generally modeled via supervised learning, detecting events via IOB sequence models and assigning event classes and attributes with multi-class classifiers. The input can be neural models starting from encoders; or classic feature-based models

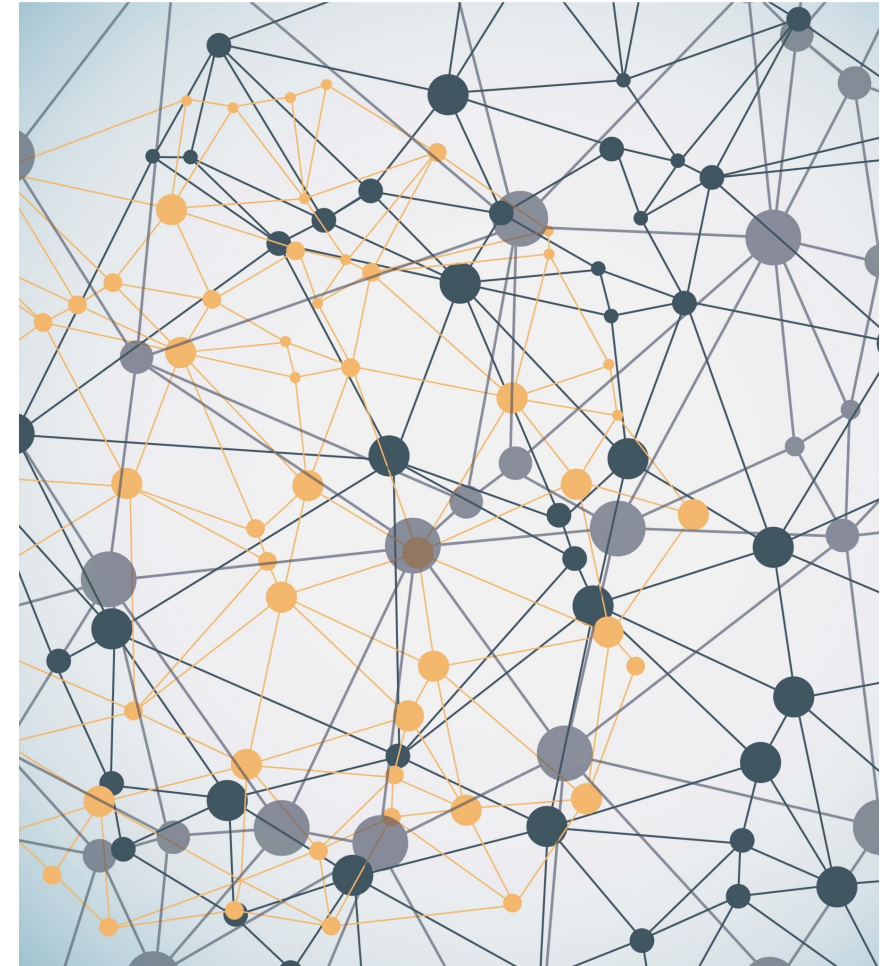
Feature	Explanation
Character affixes	Character-level prefixes and suffixes of target word
Nominalization suffix	Character-level suffixes for nominalizations (e.g., <i>-tion</i>)
Part of speech	Part of speech of the target word
Light verb	Binary feature indicating that the target is governed by a light verb
Subject syntactic category	Syntactic category of the subject of the sentence
Morphological stem	Stemmed version of the target word
Verb root	Root form of the verb basis for a nominalization
WordNet hypernyms	Hypernym set for the target

Features commonly used in classic feature-based approaches to event detection.

Natural Language Processing

Chapter 8: Question Answering

PhD. Nguyen Thi Mai Trang



Content

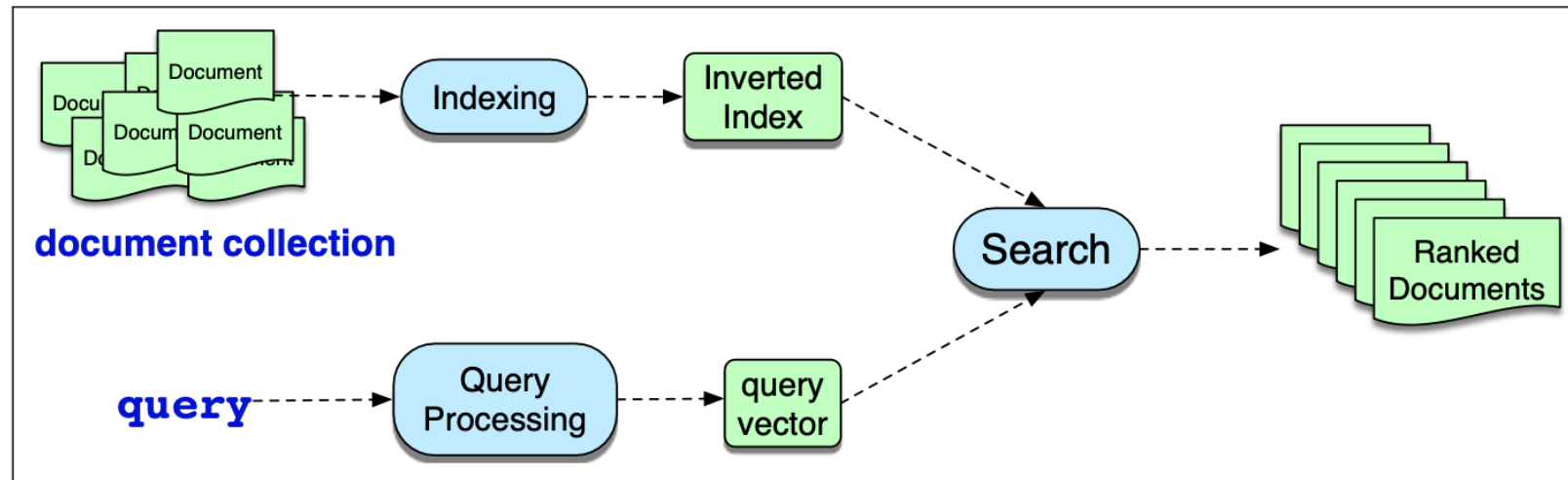
- Introduction to question answering
- IR-based factoid question answering
 - Question processing
 - Query formulation
 - Answer types
 - Document and passage retrieval
 - Answer extraction
- Knowledge-based question answering
 - Rule-based methods
 - Supervised methods
 - Semi-supervised methods
- Evaluation of factoid answers

Question Answering

- **Question answering systems** are designed to fill human information needs that might arise in situations like talking to a virtual assistant, interacting with a search engine, or querying a database.
- By the early 1960s, systems used the two major paradigms of question answering
 - **information retrieval (IR) based QA** (open domain QA)
 - **knowledge-based question answering**

Information Retrieval

- **Information retrieval** or **IR** is the name of the field encompassing the retrieval of all manner of media based on user information needs.
- The resulting IR system is often called a search engine.
- The IR task we consider is called ad hoc retrieval, in which a user poses a query to a retrieval system, which then returns an ordered set of documents from some collection.



The architecture of an ad hoc IR system

IR-based Factoid Question Answering

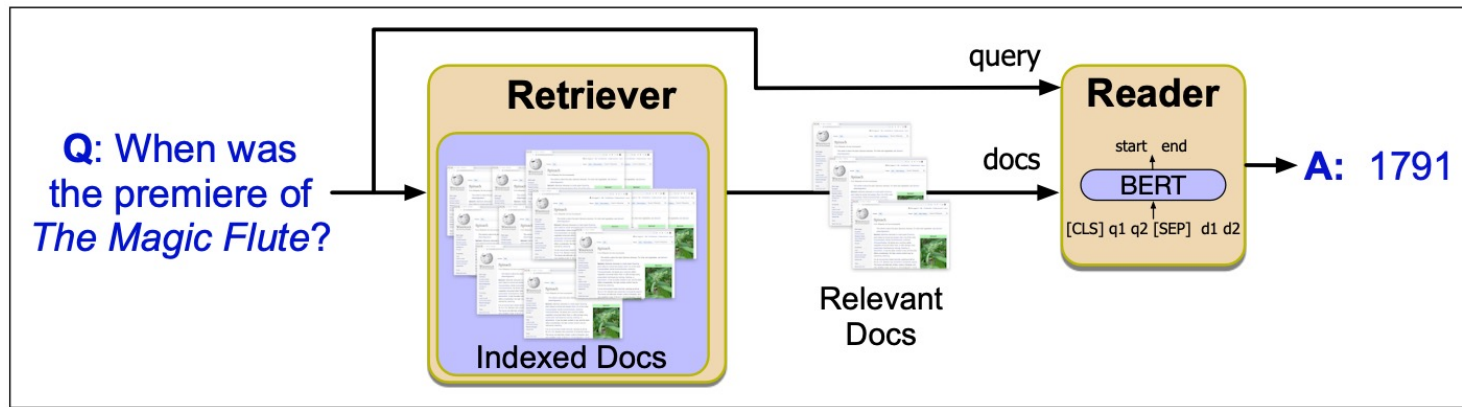
- The goal of IR-based QA (sometimes called open domain QA) is to answer a user's question by finding short text segments from the web or some other large collection of documents.

Question	Answer
Where is the Louvre Museum located?	in Paris, France
What are the names of Odin's ravens?	Huginn and Muninn
What kind of nuts are used in marzipan?	almonds
What instrument did Max Roach play?	drums
What's the official language of Algeria?	Arabic

Some factoid questions and their answers

IR-based Factoid Question Answering

- The dominant paradigm for IR-based QA is the retrieve and read model
 - In the first stage of this 2-stage model we **retrieve relevant passages** from a text collection, usually using a search engines of the type we saw in the previous section.
 - In the second stage, a neural **reading comprehension algorithm** passes over each passage and finds spans that are likely to answer the question



IR-based factoid question answering has two stages: retrieval, which returns relevant documents from the collection, and reading, in which a neural reading comprehension system extracts answer spans.

- Some question answering systems focus only on the second task, the **reading comprehension** task

Question Processing

- The first step in IR-based QA is to process the question. This involves identifying the key concepts in the question and their relationships to each other.
- The question type (e.g., *who*, *what*, *when*, *where*, *why*, *how*) is also identified, as this can help to determine the answer type.
- Ex:

What is the capital of France?

The question processing step would identify the key concepts in the question as "**France**" and "**capital**". The question type is "**what**", which indicates that the answer is **a factual entity**.

Query Formulation

- Query formulation is the process of creating a set of keywords that are used to search the document collection. The query should be as specific as possible, but it should also be broad enough to capture all of the relevant documents.
- Ex:

What is the capital of France?

The query formulation step would generate the following query: **France capital**

Answer Types

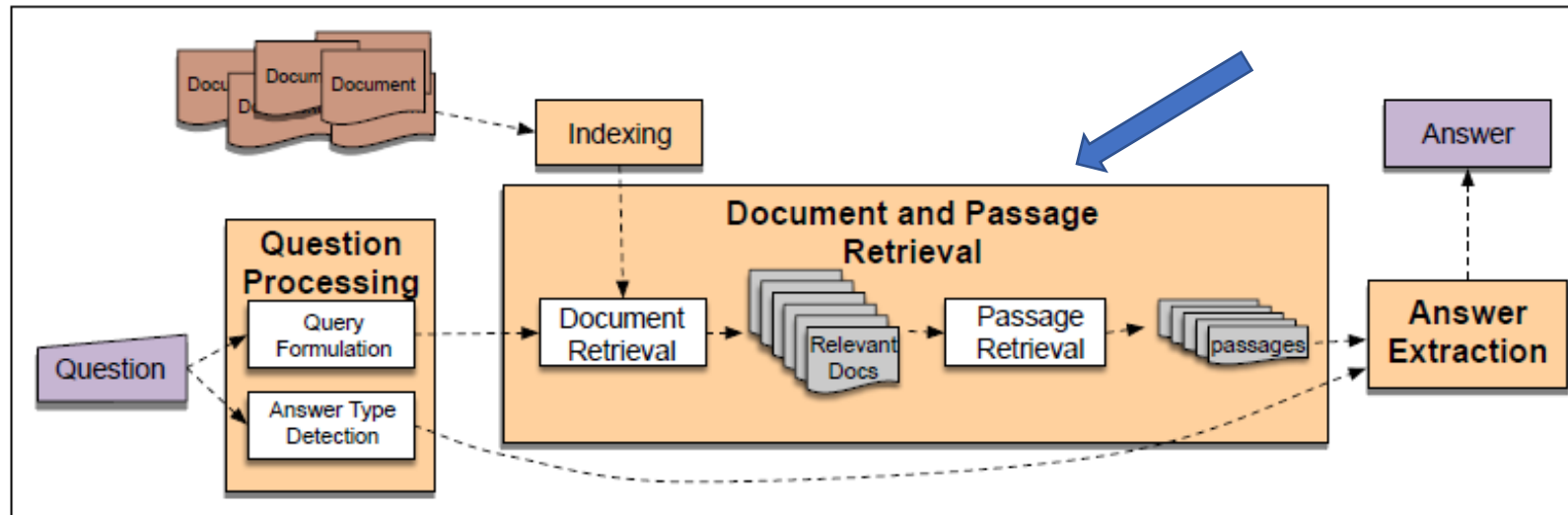
- **What are answer types?**
- There are different types of answers that can be given to questions, such as **factual answers**, **descriptive answers**, and **list answers**. The answer type can be inferred from the question type and the key concepts in the question.

• Ex:

Question	Answer	Type
What is the capital of France?	Paris	Factual answer
What is the weather like in Hanoi today?	It is sunny and 25 degrees Celsius.	Descriptive answer
What are the names of the planets in our solar system?	Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, and Neptune.	List answer

Document And Passage Retrieval

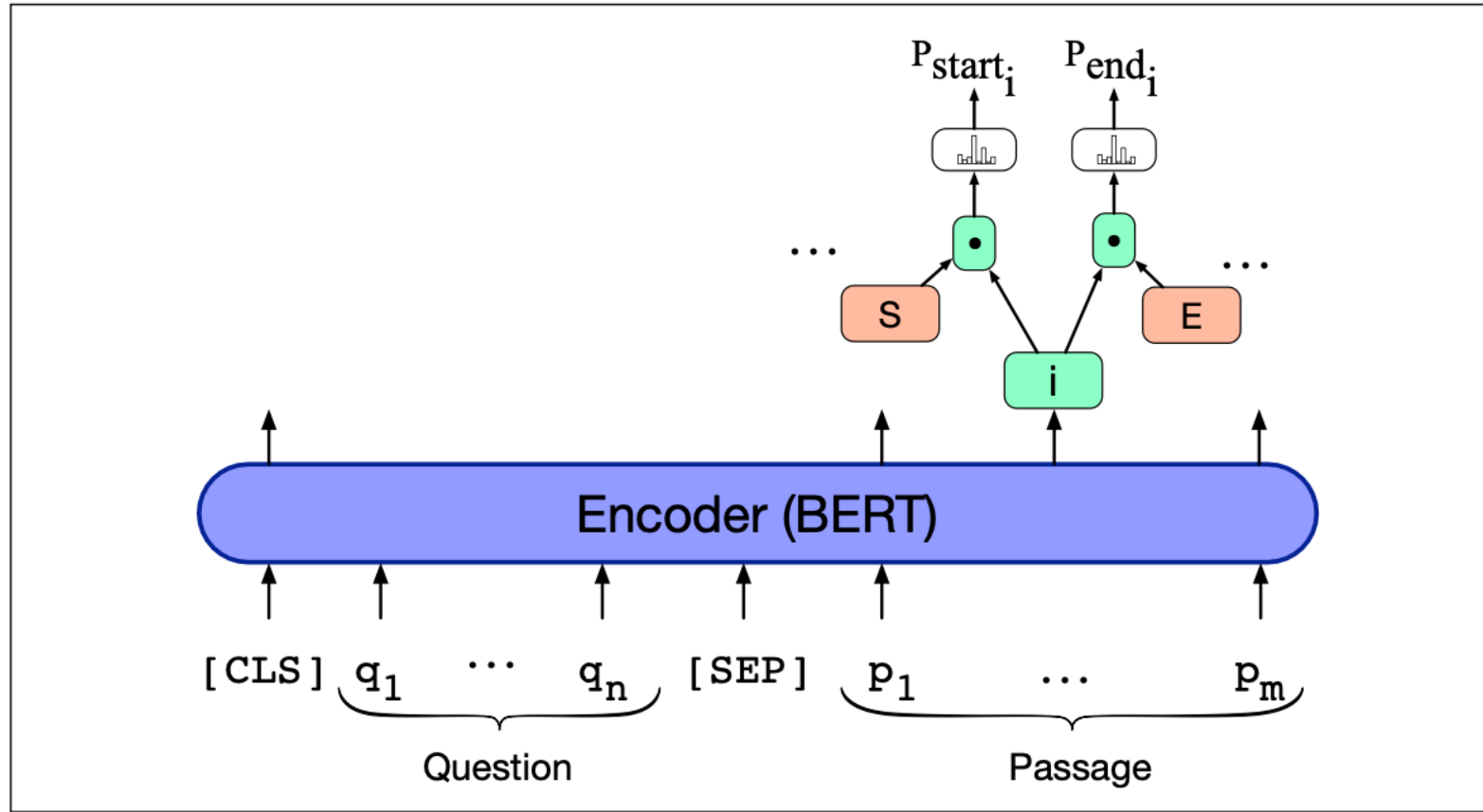
- **Document and passage retrieval** is the process of retrieving the most relevant documents and passages from the document collection. This is done by using a **search engine** to match the query keywords to the words in the documents. The documents are then ranked based on their relevance to the query.
- The top-ranked documents are then split into passages. Passages are typically sentences or paragraphs that contain a single, coherent piece of information. The passages are then ranked based on their relevance to the question and their likelihood of containing the answer.



IR-based QA: Reader (Answer Span Extraction)

- The first stage of IR-based QA is a **retriever** and the second stage is the **reader**. The reader's job is to take a passage as input and produce the answer. The answer is a **span of text** in the passage.
- The answer extraction task is commonly modeled by **span labeling**: identifying in the passage a span (a continuous string of text) that constitutes an answer.
- Neural algorithms for reading comprehension are given a **question** q of n tokens $q_1, \dots, q_n$ and a **passage** p of m tokens $p_1, \dots, p_m$. Span a is the answer.
 - Their goal is to compute the probability $P(a|q, p)$ that each possible span
 - If each span a starts at position a_s and ends at position a_e , the assumption is that this probability can be estimated as $P(a|q, p) = P_{\text{start}}(a_s|q, p)P_{\text{end}}(a_e|q, p)$

IR-based QA: Reader (Answer Span Extraction)



An encoder model (using BERT) for span-based question answering from reading-comprehension-based question answering tasks.

Knowledge-based Question Answering

- Knowledge-based question answering (KBQA) is a type of QA that uses a knowledge base (KB) to answer questions. A KB is a structured collection of knowledge, such as **facts, relationships, and rules**. KBQA systems use the KB to understand the meaning of the question and to generate an answer.
- **Two common paradigms** are used for knowledge-based QA.
 - Graph-based QA, models the knowledge base as a graph, often with entities as nodes and relations or propositions as edges between nodes.
 - QA by semantic parsing, using the semantic parsing methods

Rule-based Methods

- Rule-based KBQA systems use a set of rules to answer questions. The rules are typically hand-crafted by experts and are designed to capture the knowledge and reasoning required to answer a specific type of question.
- Consider the following rule:
IF the question is "What is the capital of [country]?" **THEN** the answer is the value of the capital slot for the entity [country] in the KB.
This rule could be used to answer a variety of questions, such as "What is the capital of France?", "What is the capital of Japan?", and "What is the capital of Canada?"

Supervised Methods

- Supervised KBQA systems are trained on a dataset of question-answer pairs. The model learns to map questions to answers by using the knowledge and reasoning contained in the KB.
- Consider the following question-answer pair:
Question: What is the capital of France? **Answer:** Paris
The model would learn that the answer to the question "What is the capital of [country]?" is the value of the capital slot for the entity [country] in the KB.

Semi-supervised Methods

- Semi-supervised KBQA systems use a combination of labeled and unlabeled data to train the model. The labeled data is used to learn the basic relationships between entities in the KB. The unlabeled data is used to refine the model's understanding of the KB and to learn new relationships between entities.
- Consider a semi-supervised KBQA system that is trained on the following labeled data:
Question: What is the capital of France? **Answer:** Paris
Question: What is the capital of Japan? **Answer:** Tokyo
The model would learn that the answer to the question "What is the capital of [country]?" is the value of the capital slot for the entity [country] in the KB.
- The system could then be used to answer unlabeled questions, such as "What is the capital of China?". The system would use its knowledge of the KB to generate a candidate answer, and then use the unlabeled data to rank the candidate answers and select the most likely answer.

Advantages And Disadvantages Of Different Methods

Rule-based KBQA systems are simple to implement and can be very accurate for answering questions that are covered by the rules. However, it can be difficult to develop and maintain a set of rules that can answer all possible questions.

Supervised KBQA systems are more flexible than rule-based systems and can be used to answer a wider range of questions. However, they require a large amount of labeled data to train the model.

Semi-supervised KBQA systems can be used to train a model on a smaller amount of labeled data than supervised methods. However, they may **not be** as accurate as supervised methods for questions that are not covered by the labeled data.

Evaluation of Factoid Answers

- Factoid question answering is commonly evaluated using **mean reciprocal rank (MRR)**.
- MRR is designed for systems that return a short ranked list of answers or passages for each test set question, which we can compare against the (human-labeled) correct answer.
- MRR is then defined as

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Each test set question is scored with the reciprocal of the **rank** of the first **correct** answer. For a system returning ranked answers to each question in a test set Q .

- Reading comprehension systems on datasets like SQuAD are evaluated via two metrics ([Rajpurkar et al., 2016](#)):
 - Exactmatch: The % of predicted answers that match the gold answer exactly.
 - F1 score: The average word/token overlap between predicted and gold answers. Treat the prediction and gold as a bag of tokens, and compute F1 for each question, then return the average F1 over all questions.